Next Gen Tire Inflation

Manual for version 1.3.4 (beta), November 2026

Part 1. For dummies

Getting started

1. What it does
2. What you need
3. Install
4. Your first drive
5. The settings worth knowing
6. When something is wrong

Feature by feature

7. Pressure modes and the pressure scale
8. Grip by surface
9. Flattening and how a flat tire rolls
10. Fuel and steering
11. Rolling drag
12. Tire damage
13. Changing only while standing still
14. The outside temperature
15. Hard mode
16. Globalhacker insanity mode
17. Vanilla balance
18. Base grip and the asphalt floor
19. The panel, the keyboard and the pad
20. Sounds
21. With other mods
22. How the mod finds its way into the game
23. Every setting in the ini
24. Reading the log

Part 2. Technical reference

25. How the mod runs
26. The pressure scale
27. How fast the pressure moves
28. Grip: what the game has and what the mod writes
29. Flattening
30. How a flattened tire rolls
31. Fuel and steering
32. Rolling drag
33. Tire damage
34. The outside temperature
35. Hard mode and the insanity mode: the full numbers
36. The vanilla balance
37. Getting into the game: the loader
38. Finding the game's code
39. The memory map
40. The game-thread hook
41. Drawing
42. The pad
43. Shared memory
44. Sounds
45. Limits and known issues
46. The offline tests
47. Constants at a glance
48. Licence and credits

Part 1, for dummies, says what the mod does and how to use it, first the basics and then feature by feature. Part 2, the technical reference, gives the models behind it: the formulas and constants and where each number comes from, how the mod gets into the game and reads and writes its memory, and what it cannot do.

Part 1. For dummies

Getting started

1. What it does

The truck you drive gets tire pressure you can change while you play, as in Expeditions: A MudRunner Game. Let air out and the tires flatten: they grip harder in dirt, sand, on rock and in mud, but lose grip on asphalt, and the truck burns more fuel and steers slower. Pump them up for the road. Drive too fast on soft tires and they wear out and go flat.

Mode	Use it for	Keep under
Low	Deep mud, sand, rocks, crawling	20 km/h
Reduced	Forest tracks, mixed ground	35 km/h
Normal	Everything else, as the game ships it	no limit
Increased	Roads: more grip on asphalt, less fuel	no limit

2. What you need

- SnowRunner for Windows. The mod is made on the Steam version. The Epic Games Store and Game Pass versions have not been tested: the mod checks the game itself at every start, and where it cannot work it does nothing and says so in its log.
- ReShade 6.8.0, which comes in the zip as `dxgi.dll` and `ReShade.ini`, for compatibility with overlays such as RivaTuner, Discord, Nvidia and Steam. It shows nothing of its own; Home opens its window. A ReShade already in the game works as it is (any ReShade from 6.8.0 on with add-on support). Without ReShade the game can crash with the RivaTuner or Discord overlay or with multiple monitors, and the mod says so at the start.

3. Install

1. Close the game.
2. Open the game's `Sources\Bin` folder, where `SnowRunner.exe` is. In Steam: right click SnowRunner, Manage, Browse local files, then `Sources\Bin`.
3. Copy the files from the zip there: `version.dll`, `TirePressure.asi`, `dxgi.dll` and `ReShade.ini`.
4. Start the game. F2 opens the NEXT GEN menu, which has a Tire Inflation tab.

On Linux (Proton, the Steam Deck) set SnowRunner's launch options in Steam (right click, Properties, General) to `WINEDLLOVERRIDES="version=n,b" %command%`. Then the game takes this `version.dll` instead of Wine's own. The mod there is not yet tested by a player.

> **Already using ReShade?** Choose Skip for `dxgi.dll` and `ReShade.ini` when Windows asks, and yours stays. The mods draw without ReShade too. It is in the zip for RivaTuner users, whose game does not boot without it. With Spotter installed, the overlay draws through Spotter's frame hook; `NextGen.ini` `[Overlay]` `Spotter=0` turns it off. **Already using other mods?** If other `.asi` mods already work in that folder, you have an ASI loader: copy only `TirePressure.asi`. If another mod's `version.dll` is there and it is not an ASI loader, rename that one to `version_chain.dll` first; this loader passes everything on to it. To update, copy the new `TirePressure.asi` over the old one, and keep only one `.asi` file of this mod in the folder. Coming from 1.2 with the ReShade it brought: this zip's `dxgi.dll` can go over it.

4. Your first drive

1. Stop the truck. By default the pressure only changes while the truck stands still.
2. On a pad, hold LB and press the d-pad down: the panel opens. Pick a mode with the d-pad left or right and confirm with A. On the keyboard, press F3: it opens the panel and steps the pressure down; leave it alone for five seconds and the choice confirms itself.
3. Listen: the air goes out (or in) for up to six seconds, and the tires flatten.
4. Drive. If the red **TIRE DAMAGE** warning shows at the top left, you are too fast for the pressure: slow down or pump the tires up.

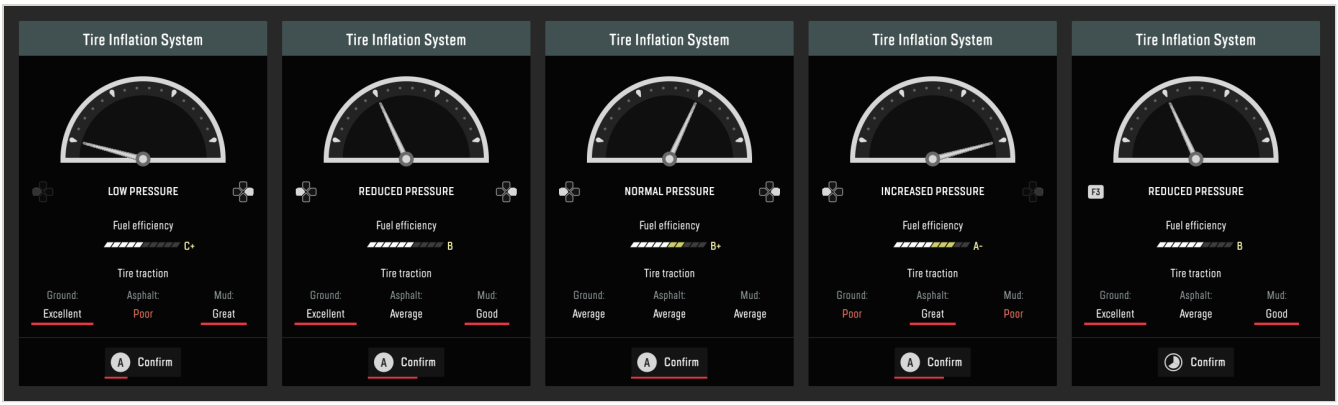


Figure 1. The panel in Low, Reduced, Normal and Increased, and as the keyboard shows it.



Figure 2. The warning while the truck is too fast for its tires, with the most worn tire's damage.

5. The settings worth knowing

All of them are in the Tire Inflation tab of the NEXT GEN menu and save themselves into `TirePressure.ini`. F2 opens and closes the menu; to give it another key, click the key's name at the menu's top right and press the new one. The key is kept in `NextGen.ini`, which the Next Gen mods share, and while the menu is open the game gets no mouse and no keys, and the panel's key and pad buttons rest. With Next Gen Graphics installed the menu also has a Graphics tab. It switches Graphics' effects while you play. Its F keys are off then; KeysWithMenu=1 in SnowRunnerShadows.ini keeps them.

- **Hard mode**: the numbers and options nearest to real tires, all at once. Everything else is greyed out while it is on.
- **Globalhacker insanity mode**: harder than real tires, for players who want the game to fight back.
- **Pressure changes only while the truck stands still**: on by default. Switch it off to change pressure while driving.
- **Tires feel the outside temperature**: on by default. Cold air means softer tires and cold rubber grips less on the road. With Next Gen Gameplay running, the temperature follows the map and the hour.
- **Vanilla balance**: for modded trucks with tires that grip better than anything in the game. It brings them down to the vanilla level, or below with a strength over 1.
- **Sound volume, the beep, the panel key and the panel's size.**

6. When something is wrong

- Open `TirePressure.log` in `Sources\Bin`. If an early line says "standing down", the mod could not work with your build of the game and changed nothing: send that log to the author.
- When another program holds the game's swap chain in a way the mod cannot draw on, `TirePressure.log` names that program.
- No panel and no menu: the "overlay:" lines in `TirePressure.log` say why. A minimap mod with a ReShade add-on mode of its own needs that mode on next to ReShade: without it the game quits while loading.
- The game crashes without ReShade with the RivaTuner or Discord overlay or with multiple monitors: put ReShade in from the zip (Install, step 3). With ReShade the same PCs ran fine.
- Support, discussion and ideas: the author's channel on EnthusiastRunner's Discord, discord.com/invite/MssDb2Pvcj, or an issue on GitHub for a bug report; `GrabDumps.bat` from the zip gathers the logs, the ini files and the crash dump into one zip for it.
- "key pressed, no driven truck found" in the log: you were not driving a truck.
- To remove the mod, delete `TirePressure.asi`, `TirePressure.ini`, `TirePressure.log` and `TirePressure.prev.log` from `Sources\Bin`. `version.dll` and `AsiLoader.log` go too when no other `.asi`

mod is left (Next Gen Gameplay loads through them). If you renamed another mod's `version.dll` to `version_chain.dll`, rename it back. `NextGen.ini` goes too when no other Next Gen mod is left. ReShade from the zip goes the same way when nothing else uses it: `dxgi.dll`, `ReShade.ini` and `ReShade.log`.

7. Pressure modes and the pressure scale

The numbers are multipliers against the tire's own values from its file; Normal is the tire as the game ships it.

Table 1. The modes as they come

	Low	Reduced	Increased
Grip on dirt	×3.5	×3	×0.9
Grip on gravel	×2.75	×2.4	×0.95
Grip on sand	×4	×3.4	×0.85
Grip on rock	×3.5	×3	×0.9
Grip on asphalt	×0.85	×0.95	×1.3
Grip in mud	×1.3	×1.15	×0.85
Tire radius, up to	10 cm less	6 cm less	unchanged
Fuel use while moving	×1.5	×1.25	×0.85
Steering speed	×0.7	×0.85	×1.1
Wear starts above	20 km/h	35 km/h	no wear
Most wear from	30 km/h	45 km/h	
Damage per wheel	4 to 8	2 to 6	
Every	7 seconds	10 seconds	

The pressure itself is one number on a scale, and every mode is a point on it: Low 45%, Reduced 65%, Normal 100% and Increased 125% of Normal's pressure. Every factor is read off the scale straight between the two nearest modes, so a change walks through all the values in between instead of jumping. Normal to Low takes `Seconds` (6 by default) and a smaller step its share: Normal to Reduced about two thirds of it. A choice changed half way turns round where the pressure stands.

The mode belongs to the truck you drive and stays with it (*Each truck keeps its own pressure*, below). A truck not driven before starts in the highest mode on offer. `Increased=0` leaves the road mode out. Every number in the table is editable in the tab, per mode.

8. Grip by surface

A SnowRunner tire file has three grip numbers: `BodyFriction` for plain ground, `BodyFrictionAsphalt` for paved ground and `SubstanceFriction` for mud. The mod gets six surfaces out of them from how the game applies them.

- Every frame the game marks each wheel with what lies under it: paved, gravel, sand, hard (rock), and the soil and mud layers. The mod reads those marks 20 times a second.
- The game's contact code sets the wheel's physical friction from one of the tire's numbers: on paved ground from the asphalt value on the wheel, otherwise from a per-wheel "ground grip" list on the truck that holds each tire's `BodyFriction`, and in mud from the mud value.
- The mod writes the inputs, never the physical friction, which the game rewrites every frame. Into each wheel's slot of the ground list it puts the tire's own `BodyFriction` times the mode's factor for what is under that wheel right now: dirt, gravel, sand or rock. Asphalt and mud get their own factors on the wheel. Gravel, sand and rock are not separate game values; the mod swaps the one ground number per wheel as the ground under it changes.

- Each wheel is on its own: a truck with the left side on a gravel road and the right side in sand grips differently left and right. Ice is left to the game, which has its own ice friction per wheel. Each grip multiplier stops at 10, as in Expeditions, and so does the ground grip the game is given.

9. Flattening and how a flat tire rolls

A lower pressure shrinks the wheel's collision cylinder, and the game's own soft tire rendering draws the flattening. The game's wheel shader can only draw a tire so flat before the tread sinks into the ground, and that limit depends on the tire's size. The mod works it out from the shader's own formula:

Table 2. How flat the game can draw a tire

Tire radius	0.35 m	0.47 m	0.65 m	0.80 m	1.00 m
Can be drawn flatter by	7.1 cm	8.6 cm	10.2 cm	10.8 cm	10.3 cm

The tire's own offset from its file takes part of that room. Where the deepest mode asks for more than a tire has, every mode's flattening shrinks by the same share, so the deepest mode takes all the room: a scout on 94 cm tires gets 6.7 cm at Low and 4 cm at Reduced. The cylinder always keeps at least 40% of the wheel's radius.

Tires that come soft. Some modded tires sink into the ground under the truck's weight: their contact sits inside the round shape the game collides with, and the game draws a tire by where it touches the ground, so such a tire already looks flatter at Normal. The mod measures that sink under each wheel, smoothed over a couple of seconds, and takes it off the room first: a soft tire is flattened less, and no tire is drawn into the ground. A vanilla tire sinks next to nothing and is flattened as the table says. The tab's truck section shows the first wheel's sink and the room left. In the cold no mode goes past the room either.

A real tire's belt keeps its length, so a flattened tire rolls nearly as far per turn as before. In the game the wheel *is* the collision cylinder, so a flattened tire would lose about three times the speed a real one does. With **Flattened tires roll like real ones** (`RollingRadius=1`, the default) the mod raises the truck's gear speeds by the difference: that scout loses about 5% of its speed at Low and 3% at Reduced instead of 15% and 9%. Worn out tires are left out of that sum.

10. Fuel and steering

The fuel factor scales the truck's engine and gearbox `FuelConsumption`, and only counts while the truck moves: standing at idle burns as usual. The steering factor scales the truck's steering speed, so soft tires turn slower.

11. Rolling drag

One checkbox in the Realism section, or `RollingDrag=1`. It is on by default; untick it to turn it off. Hard mode and the insanity mode keep it on. A tire rolls harder on soft ground and when it is aired down. The game slows a rolling truck by itself, but its tires roll the same at every pressure. With this on the mod adds a rolling resistance by the ground and the pressure.

The resistance is a force of Crr × N on each wheel on the ground. N is the wheel's share of the truck's weight. Crr goes by the ground under the wheel:

Table 3. Rolling drag by ground

Ground	Crr	Slows the truck at Normal, m/s²	At Low, m/s²
Paved	0.010	0.10	0.15
Rock	0.015	0.15	0.22
Gravel	0.020	0.20	0.29
Dirt, mud and no flag	0.030	0.29	0.44
Sand	0.060	0.59	0.88
Ice	0	0	0

The pressure scales it by $(p / p_{Normal})^{-0.5}$: Low ×1.49, Reduced ×1.24, Normal ×1, Increased ×0.89. It follows the pressure through a change. The numbers are the mod's own, from published truck tire figures (0.006 to 0.01 on asphalt at the rated pressure).

- The game shows mud to the mod as no ground of its own. So mud gets dirt's number, on top of the game's own drag in mud. The mod cannot yet tell the two apart.
- The weight is the truck's as the game's physics has it, its gravity included. Cargo and a trailer are not counted.
- The drag goes in through each wheel's own damping, the way the game drags a flat tire. It acts with the foot off too. A broken wheel keeps the game's own.
- It needs the game's truck update, as tire damage on the game's thread does. On a build where the mod does not find it, rolling drag stays off and the log says so.
- Fuel: the engine works harder against the drag and burns more by itself. So while rolling drag is on, every mode's fuel factor is 1, the default modes' too. Their Fuel use rows keep their values, which count again with rolling drag off.

12. Tire damage

While the truck is faster than the mode's first speed, a timer runs. Each time it passes the mode's interval, every wheel on the ground takes damage: the least at the first speed, rising to the most at the second speed and above. Below the first speed for a full second, the timer starts over. Increased and Normal never wear.

- A wheel takes about 50 damage; the exact number is the wheel's own. The warning shows the most worn tire in percent, with a bar.
- A worn out tire goes flat. It is the game's own wheel damage: the HUD's wheel icon turns red, the damage stays in the save, and a garage or a service repairs it like any other.
- A worn out tire keeps **a slight grip**, `WornGrip` percent of its own on the ground, on asphalt and in mud (10 by default, so the truck still moves; 0 gives none, as from 1.2), and is **drawn flat**, as flat as the game can draw it, until it is repaired.
- The mod deals the damage on the game's own thread, at the point in its frame where the game deals damage itself, and runs the game's damage update after it. On a build of the game where it does not find that place, it deals the damage from its own thread and makes the tire flat itself; the game's damage display then catches up at the next damage the game deals.
- The outside temperature moves the two speeds (section 14).

13. Changing only while standing still

With `StandStill=1` (the default) the pressure only moves while the truck stands, below about 1 km/h. A choice made while driving waits, and a change under way pauses and goes on when the truck stops again. While it waits, an amber hint "TIRE PRESSURE: Stop the truck to change the pressure" shows at the top left.

Each truck keeps its own pressure. With `PerTruck=1` (the default) the mode chosen for a truck is saved in `TirePressure.trucks.ini` next to the exe, with a line such as `Mode=Low`. One owned truck's goes by the game's

own id for it, in a section `[kind:{id}]` with `Own=1`; a kind's goes in `[kind]`. The mode in force is the truck's own, else its kind's, else the start mode. The moment that truck is driven again its mode comes back, and the pressure with it: no sound, no compressor, no wait. It holds across map loads and game starts. A truck not driven before starts in the highest mode on offer: Increased, or Normal with Increased off. Two trucks of one kind each keep their own. **This truck only**, under the checkbox, picks the level. Ticked, a mode chosen is this truck's alone. Unticked, it goes to the kind's section, and the truck's own stays with `Own=0` until it is ticked again. It is greyed out while the game gives no id for the truck, which then goes by its kind. When the game makes a truck anew under another id (a recovery, a garage), the new id takes the old one's mode over, as Next Gen Gameplay does, and the log says so in one line. The old id's section stays, marked `TakenBy` and `TakenAt`, and only the newest 10 marked sections of the file are kept. A file from 1.3.1 to 1.3.3 reads as before. With `PerTruck=0` one mode serves every truck, and every game start begins at Normal. The checkbox is in the tab's Realism section, and the tab's truck section names the kind.

14. The outside temperature

With `Temperature=1` (the default) the outside temperature acts three ways. All the modes' numbers are set for 15 °C.

- **The air.** It follows the gas law on absolute pressure, with Normal taken as 35 psi on the gauge (49.7 psi absolute). Cold air is less pressure, and since every factor is read off the pressure scale, a cold tire slides toward the next mode down: at minus 20, Normal sits halfway between Reduced and Normal and gets about half of Reduced's grip on soft ground and half its flattening. Heat slides it up toward Increased, even when the Increased mode is switched off.
- **The rubber.** Cold rubber hardens and grips less, most on hard surfaces (asphalt and rock); warm rubber grips a little better on them.
- **The wear.** A soft tire heats its sidewalls at speed: in the heat it wears from lower speeds, in the cold from higher ones.

Table 4. What the outside temperature does

Outside	Normal holds	Low holds (of its own)	Asphalt and rock	Dirt, gravel, sand, mud	Wear speeds
-40 °C	73%	63%	×0.7	×0.85	×1.25
-30 °C	78%	70%	×0.73	×0.86	×1.25
-20 °C	83%	77%	×0.79	×0.9	×1.21
-10 °C	88%	83%	×0.85	×0.93	×1.15
0 °C	93%	90%	×0.91	×0.95	×1.09
10 °C	98%	97%	×0.97	×0.98	×1.03
15 °C (reference)	100%	100%	×1	×1	×1
25 °C	105%	107%	×1.03	×1	×0.94
35 °C	110%	113%	×1.06	×1	×0.88
45 °C	115%	120%	×1.08	×1	×0.82

The temperature is `AmbientC` in the ini (15 to begin with), or, while the Gauges mod runs, the map's and the hour's from Gauges, and then the tab's slider has no effect. It is taken in half-degree steps. The tab line "the tires hold A% of Normal's pressure, B% once the air's temperature counts" shows the mode's place on the scale (A) and where the temperature moved it (B). A separate document goes through the temperature model with full tables.

15. Hard mode

One checkbox at the top of the Realism section, or `HardMode=1`. It sets the numbers and options nearest to real tires: a change takes half a minute and only while the truck stands, the tires feel the outside temperature, soft tires wear, flattened tires roll like real ones, vanilla balance is on at strength 1, rolling drag is on, no asphalt floor and no base grip. The modes' fuel factors are 1, as the engine burns more against the drag by itself. The rest of the settings are greyed out; the ini keeps your own numbers for when it is off.

Table 5. Hard mode

	Low	Reduced	Increased
Grip on dirt	×1.6	×1.3	×0.92
Grip on gravel	×1.3	×1.15	×0.95
Grip on sand	×1.9	×1.45	×0.85
Grip on rock	×1.6	×1.3	×0.92
Grip on asphalt	×0.8	×0.9	×0.97
Grip in mud	×1.35	×1.18	×0.9
Tire radius, up to	5 cm less	2.5 cm less	unchanged
Fuel use while moving	×1	×1	×1
Steering speed	×0.8	×0.9	×1.05
Wear starts above	30 km/h	50 km/h	no wear
Most wear from	55 km/h	75 km/h	
Damage per wheel	2 to 10	1 to 6	
Every	8 seconds	10 seconds	

Airing a tire down to half its pressure lengthens its contact patch by about half: that much more grip on sand and soft ground, a little less on hard ground. Over-inflating does the reverse and gains nothing on asphalt. A soft tire's sidewalls flex and heat at speed, so its wear starts higher up than the defaults but ends a tire faster.

16. Globalhacker insanity mode

One checkbox under hard mode, or `InsaneMode=1`; it excludes hard mode and greys out the same settings. It is built from the own settings of the player Globalhacker and is harder than real tires: a change takes 25 seconds and only while the truck stands, the tires feel the temperature, flattened tires lose the speed the smaller wheel loses, there is no Increased mode, the vanilla balance runs at strength 2, and rolling drag is on. The fuel factors are 1, as in hard mode.

Table 6. Globalhacker insanity mode

	Low	Reduced
Grip on dirt	×2	×1.5
Grip on gravel	×1.6	×1.3
Grip on sand	×2.3	×1.7
Grip on rock	×2	×1.5
Grip on asphalt	×0.4	×0.45
Grip in mud	×1.5	×1.25
Tire radius, up to	12 cm less	6 cm less
Fuel use while moving	×1	×1
Steering speed	×0.6	×0.75
Wear starts above	15 km/h	20 km/h
Most wear from	20 km/h	25 km/h
Damage per wheel	8 to 16	4 to 12
Every	2 seconds	2 seconds

> These numbers were made for tire files already cut by 45% on the ground and in mud. On vanilla tires they are milder than they look: Low ends near vanilla grip, not far above it. The next version also sets the base grip on the ground and in mud to ×0.55 in this mode, so it plays as its author plays it.

17. Vanilla balance

Some modded trucks come with tires that grip better than any tire in the game on the ground, on asphalt and in mud at once. The vanilla balance brings such a tire down to the vanilla level.

- The yardstick is the **vanilla envelope**: of the game's 441 vanilla truck tires (94 different grip sets), the 16 that no other tire matches or beats in all three numbers at once. It is read from the game's stock files, so tire edits made by other mods do not move it.
- A tire is cut by the smallest factor that lets some vanilla tire match or beat it in all three. Vanilla tires are inside the envelope and never change. A tire strong in one direction only loses little; one stronger than every vanilla tire everywhere ends level with the best vanilla all-rounder.
- **Strength** 0 to 1 goes from none to all the way to the vanilla level. Above 1 the cut goes on: the tire's ratio to the vanilla tire is applied again, so at 2 a tire ends as far below the vanilla tire as it started above it. It never reaches zero, and vanilla tires stay as they are at any strength.
- The tab's line "Vanilla balance makes that ×k: ground ..., asphalt ..., mud ..." shows the result for the truck you drive while you move the slider.

Table 7. The vanilla balance at different strengths

Tire (ground / asphalt / mud)	Strength 0.5	1	1.5	2
A strong modded tire (4.5 / 5 / 4)	×0.66 2.98 / 3.31 / 2.65	×0.32 1.46 / 1.62 / 1.30	×0.22 0.97 / 1.08 / 0.86	×0.11 0.48 / 0.53 / 0.42
A modded Ural (3.55 / 3.5 / 2.8)	×0.73 2.60 / 2.56 / 2.05	×0.46 1.65 / 1.63 / 1.30	×0.34 1.21 / 1.19 / 0.95	×0.22 0.77 / 0.75 / 0.60
A mud tire, strong in mud only (2 / 0.6 / 5)	×0.92 1.83 / 0.55 / 4.58	×0.83 1.67 / 0.50 / 4.17	×0.76 1.53 / 0.46 / 3.82	×0.69 1.39 / 0.42 / 3.47
Any vanilla tire (2.2 / 2.6 / 1.3)	×1 2.20 / 2.60 / 1.30	×1 2.20 / 2.60 / 1.30	×1 2.20 / 2.60 / 1.30	×1 2.20 / 2.60 / 1.30

> Mods that *raise* vanilla tires' grip clash with the balance: a vanilla tire made stronger than the stock envelope gets cut back, in all three numbers. Mods that lower vanilla tires are fine.

18. Base grip and the asphalt floor

Base grip (`BaseGround`, `BaseAsphalt`, `BaseMud`) scales every mode's grip, Normal too, on top of the vanilla balance: a quick way to make every truck grip less (or more) without touching the mode table. Base ground covers dirt, gravel, sand and rock.

The **asphalt floor** (`AsphaltFloor`, 0 = off) lifts slippery tires to at least that much grip on paved ground, as if their tire file said so; the modes act on top. Vanilla off-road and mud tires have 0.4 to 1.2 on asphalt, highway tires about 3. The idea is Naybour's.

19. The panel, the keyboard and the pad

Table 8. Controls

What	Pad (default)	Keyboard
Open the panel	LB + d-pad down	F3
Lower the pressure	d-pad left (with the panel open)	F3 steps down, from Low round to the highest mode
Raise the pressure	d-pad right (with the panel open)	
Confirm	A	leave it: it confirms itself after 5 s
Close unchanged	B	

- While the panel is open the d-pad works without LB, as in Expeditions, and the mod hides the plain d-pad and the panel's buttons from the game. LB + d-pad left or right is never the panel's, open or closed, so a gear on that chord keeps working. In menus and outside a truck the pad's buttons are the game's alone. While the game's function menu is up, and for a quarter second after it closes, the panel's pad and key controls wait and the mod hides nothing from the game.
- Every binding can be set in the tab: press its button, hold the pad buttons, let go. Any pad Windows sees as an Xbox pad works.
- `ConfirmSeconds` (5) is how long a choice waits before it applies itself; 0 means only the pad's confirm. The keyboard has no confirm key, so it needs this.
- **Show the panel** off (`UI=0`): the key changes the pressure at once, without the panel. With the beep on, the mod beeps once for Normal, twice for Reduced, three times for Low and once high for Increased.

With a pad, the triggers are shaped before the game reads them: a trigger's share is raised to the power set by **Pad throttle curve** and **Pad brake curve** in the tab's Controls section (`PadThrottleCurve`, `PadBrakeCurve`, 1 to 3, 2 by default). At 2 half the trigger gives a quarter and the full pull stays full, so a light pull is gentle; at 1 the game reads the trigger as it is. It works only while a truck is driven, so menus see the pad as it is.

20. Sounds

- Deflating plays the air being let out, weaker as the tires empty, and stops with a puff. Filling plays the air going in. Both are recordings of a real tire made for this mod.
- The compressor does not run at every filling. The air comes from a tank: the compressor fills it again once it has fallen far enough, at every second filling from Low to Normal and every fourth of a single step. It then runs for about 17 seconds and lets its own air go as it stops. The compressor is made by the mod's code; the air it lets go is a sound of SnowRunner itself, read from the game's sound file while it runs.
- You hear them while you drive a truck and the game's window is in front. `SoundVolume` 0 turns them off.
- `AirOutSound`, `AirInSound`, `CompressorSound` and `CompressorStopSound` put another sound in place of one of the four: a WAV file in `Sources\Bin`, or a sample from the game's `shared_sound.pak` such as

`[sound]\actors\actor_lamp_generator_loop.pcm`, or `none`. A WAV with a loop marked in it plays up to the loop, goes round in it, and plays what follows as the sound stops.

21. With other mods

- **Gauges** (the gauge cluster by the same author). Each mod publishes a small block of shared memory for the other. Gauges hands over the outside temperature by map and hour; the tire mod hands over its mode, pressure and wear, which Gauges shows as a tenth lamp: a tire with L, R, N or +, amber at Low and Reduced, green at Increased, blinking while the pressure moves, red while the tires take damage or one is worn out.
- **ASI loaders**: see section 3. A second copy of this mod under another name stands down and says so in the log.
- **Mods that edit tire files**: the modes multiply whatever values the game loaded, so they work on top. Only the vanilla balance compares against the stock values (section 17).
- The mod writes no game file, so it does not conflict with pak mods.

22. How the mod finds its way into the game

There is no list of supported builds. At every start the mod looks through the running game's code for the places it needs, by the bytes around them, and checks that the game's objects are laid out as it expects. Each place has to be found exactly once.

- **Essential:** the wheel and cylinder classes, the global that leads to the driven truck (looked for three ways), and six checks of where the game keeps the values the mod writes. Without them the mod stands down and changes nothing.
- **Optional:** the game's damage update and the truck update's call into Windows. Without them the mod runs and deals tire damage from its own thread.
- `DumpImage=1` in the ini writes the game's code as the mod sees it into `TirePressure_image_<stamp>.bin` next to the exe at the next start. That file lets the author see exactly why a build fails; take the line out again afterwards. A Game Pass install cannot be copied from its folder, but this can.
- What the mod writes into the game's memory: the wheels' asphalt and mud grip and collision radius, the truck's ground grip list, fuel consumption, steering speed and gear speeds, the wheels' damage, one entry of the game's import table (for the game-thread damage) and the game's pointers to the pad function (two in the Steam build). All reads and writes go through checked calls, so an object the game frees gives an error, not a crash.

23. Every setting in the ini

`TirePressure.ini` next to the exe is written with the defaults and a short guide on the first start. The tab edits all of it except the four sound names and `DumpImage`. Each truck's mode goes into `TirePressure.trucks.ini` beside it (section 13).

Table 9. [TirePressure]

Key	Default	Range	What it does
Key	114 (F3)	virtual key code	The panel key
Beep	0	0 or 1	Beep as a change begins
Seconds	6	0 to 60	Normal to Low takes this long
SoundVolume	50	0 to 100	0 = no sound
UI	1	0 or 1	Show the panel
UIScale	1	0.3 to 3	Panel size
ConfirmSeconds	5	0 to 30	A choice confirms itself after this; 0 = never
PadOpen, PadLower, PadRaise, PadConfirm, PadCancel	LB+DPadDown, DPadLeft, DPadRight, A, B	buttons joined with +, or None; PadLower and PadRaise without LB or RB	Pad bindings
BaseGround, BaseAsphalt, BaseMud	1	0.1 to 5	Scale every mode's grip
VanillaBalance	0	0 or 1	Vanilla balance on
VanillaBalanceStrength	1	0 to 2	How far, past 1 below vanilla
AsphaltFloor	0	0 to 10	Least grip on paved ground; 0 = off
WornGrip	10	0 to 30	Percent of its grip a worn out tire keeps; 0 = none
Increased	1	0 or 1	The road mode
TireDamage	1	0 or 1	Soft tires wear at speed
RollingRadius	1	0 or 1	Flattened tires roll like real ones
StandStill	1	0 or 1	Change only while standing
PerTruck	1	0 or 1	Each truck keeps its own pressure mode
Temperature	1	0 or 1	Tires feel the outside temperature
RollingDrag	1	0 or 1	Tires roll harder on soft ground and aired down
AmbientC	15	-40 to 50	Outside temperature without Gauges
HardMode	0	0 or 1	Hard mode
InsaneMode	0	0 or 1	Globalhacker insanity mode; wins over HardMode
AirOutSound, AirInSound, CompressorSound, CompressorStopSound	empty	WAV file, pak sample or none	Replace a sound

Key	Default	Range	What it does
DumpImage	0	0 or 1	Write the game's code image at the next start, for a report

Table 10. [Low], [Reduced], [Increased] (Expeditions' own key names)

Key	Range	What it does
OnModelFriction	0.1 to 10	Grip on dirt (plain ground)
GravelFriction, SandFriction, RockFriction	0.1 to 10	Grip on gravel, sand, rock
BodyFrictionAsphalt	0.1 to 10	Grip on paved ground
SubstanceFriction	0.1 to 10	Grip in mud
AdditionalRadiusOffset	0 to 0.25	Metres off the radius
FuelConsumptionModifier	0.1 to 10	Fuel while moving
SteeringSpeedModifier	0.1 to 10	Steering speed
MinVel, MaxVel	0 to 100	Wear speeds in metres per second (10 = 36 km/h)
MinDamage, MaxDamage	0 to 100	Damage per tick; MaxDamage 0 = no wear
DamageTick	1 to 100	Seconds between damages

24. Reading the log

Table 11. Lines worth knowing

The log says	Meaning
game code found at look N	Everything found; all is well.
the game's truck update is not where the mod knows it, so tire damage will be dealt from the mod's own thread	Runs on this build; damage is dealt by hand.
this game version is not one the mod can work on: standing down. Not as the mod needs it: ...	Nothing done. Send the log; DumpImage=1 helps further.
this mod is in the folder twice	Keep only one .asi file of the mod.
outside temperature: X C from another mod's block	The temperature comes from Gauges.
outside temperature: AmbientC X C (no other mod publishes one)	The temperature is AmbientC.
state published for other mods in Local\SnowRunnerTirePressure.State...	The block for Gauges is there.
wheel ... worn out: 10 percent of its grip until it is repaired	A tire wore out.
key pressed, no driven truck found	The key was pressed outside a truck.
panel: the overlay did not start: the key cycles the modes directly	The overlay found no swap chain; the overlay: lines say why.

Part 2. Technical reference

This part is for readers who want the workings. It gives each model as the code has it, the formulas with every constant, where a number comes from when the code says so, how the mod gets into the game and what it reads and writes there, and what it cannot do. The numbers are those of version 1.3.4. File and line references (`tire_pressure.cpp:813`) point into the mod's source. "The Steam build" is the game build of 22 July 2026 (time stamp `6a607c05`), the one the mod was made on; an RVA is an offset from the start of `SnowRunner.exe` in memory, in that build.

> **Notation.** *P* is the tire pressure as a share of Normal's, so Normal is 1. Modes are numbered Low 0, Reduced 1, Normal 2, Increased 3. A factor such as ×1.3 multiplies the game's own value. clamp(*v*, *a*, *b*) holds *v* between *a* and *b*. Temperatures are in °C, speeds in km/h unless m/s is written.

25. How the mod runs

The loader, `version.dll`, is loaded by the game at its start because Windows looks for that DLL next to the exe first. It passes every call on to the real `version.dll` and loads `TirePressure.asi`. The ASI starts one thread, `Run`, inside the game. Everything else happens on that thread, outside the game's frame: it finds the game's code once, then loops with a 50 ms sleep, about 20 passes a second (`tire_pressure.cpp:1475`). The only code that runs on the game's own thread is the damage update, through the hook described in its section.

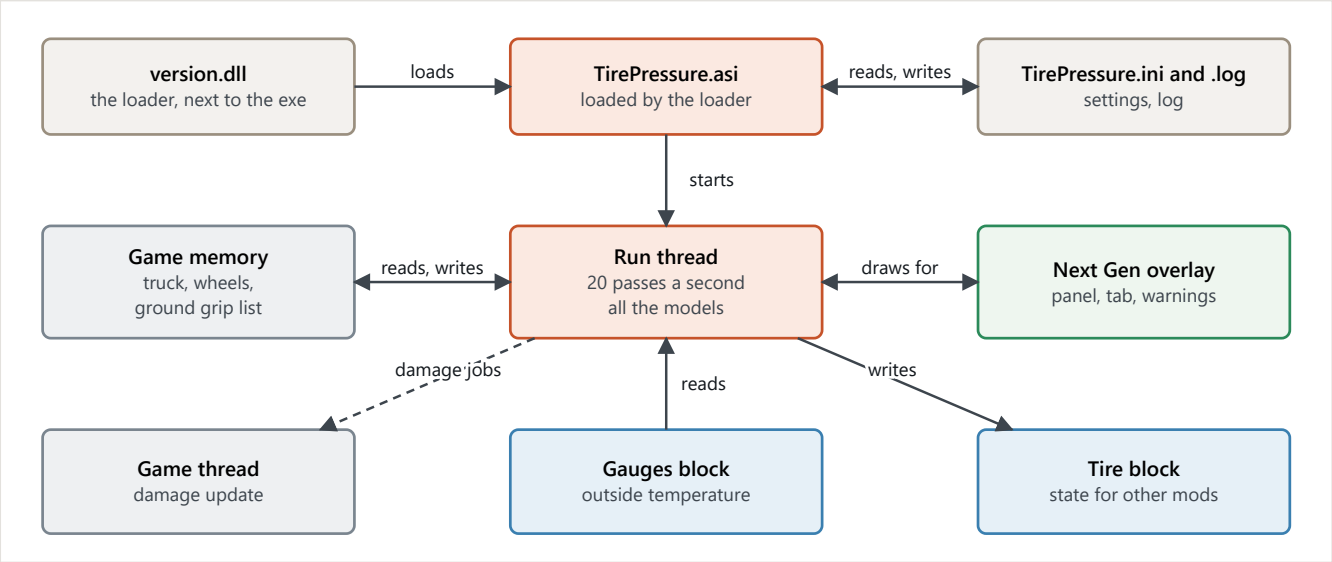


Figure 3. The parts and what flows between them. Solid arrows run every pass, the dashed one only when a tire takes damage.

Each pass with a truck, in order (`tire_pressure.cpp:1692-1838`):

1. The pressure takes one step toward the target, unless the truck must stand still and does not.
2. The air tank and the sounds follow the step.
3. The outside temperature is read: from the Gauges block when it is fresh, otherwise `AmbientC`.
4. The temperature effects are worked out for the temperature rounded to half a degree.
5. The pressure in the tires is corrected by the gas law, and the mode factors are read off the pressure scale at that pressure.
6. The rubber factors scale the grip: one for hard surfaces, one for soft.
7. Each wheel gets its vanilla balance factor, and asphalt its floor.
8. Each wheel's sink is measured, and the wheel is written: its asphalt and mud grip and its collision cylinder. A worn out tire gets WornGrip of its grip (10 percent) and a flat cylinder.

9. The speed is read, the rolling flag set, and the ground under each wheel picked.
10. The truck is written: fuel use, steering speed, the ground grip list and the gear speeds.
11. The damage timer runs, and damage is dealt when it is due.
12. The panel's view and the shared state block are updated.

Before the first truck is found the wheels are looked up every tenth pass, twice a second; from then on, every pass. The pressure step and the air tank take a fixed 0.05 s per pass (`kPass`), not the measured time, so a pass delayed by the system stretches a change a little. The damage timer uses the measured time, at most 0.5 s per pass.

26. The pressure scale

The four modes sit on one scale of pressure. Normal is taken as 35 psi gauge; the other three are the mod's choice (`panel.h:138-140`).

Table 12. The pressure scale (panel.h:140)

Mode	P	Gauge pressure	In bar	Absolute, share of Normal's
Low	0.45	15.75 psi	1.09 bar	0.613
Reduced	0.65	22.75 psi	1.57 bar	0.754
Normal	1	35.00 psi	2.41 bar	1.000
Increased	1.25	43.75 psi	3.02 bar	1.176

Every factor the mod uses (grip on each surface, flattening, fuel, steering) is read off this scale at the current pressure P by `FactorsAt` (`tire_pressure.cpp:813-834`). With $P_{0..3}$ the four modes and $f_{0..3}$ a factor's value at each:

$$\mathrm{i} = \text{the first segment with } \mathrm{P} \le \mathrm{P}_{\mathrm{i}+1}, \text{ at most } 2 \tag{1}$$

$$\mathrm{t} = \mathrm{clamp}(\ (\mathrm{P} - \mathrm{P}_{\mathrm{i}})\ /\ (\mathrm{P}_{\mathrm{i}+1} - \mathrm{P}_{\mathrm{i}}), -0.5, 1.5\ ) \tag{2}$$

$$\mathrm{f}(\mathrm{P}) = \mathrm{clamp}(\ \mathrm{f}_{\mathrm{i}} + (\mathrm{f}_{\mathrm{i}+1} - \mathrm{f}_{\mathrm{i}}) \cdot \mathrm{t}, \text{range}\ ) \tag{3}$$

The ranges are those of the game Expeditions for the same keys: 0.1 to 10 for grip, fuel and steering, 0 to 0.25 m for the radius offset (`panel.h:81-83`). The clamp on t lets the last slope go on for half a step past Low and past Increased and then stop: below P = 0.35 and above 1.375 nothing changes any more. Only the temperature can push the pressure there.

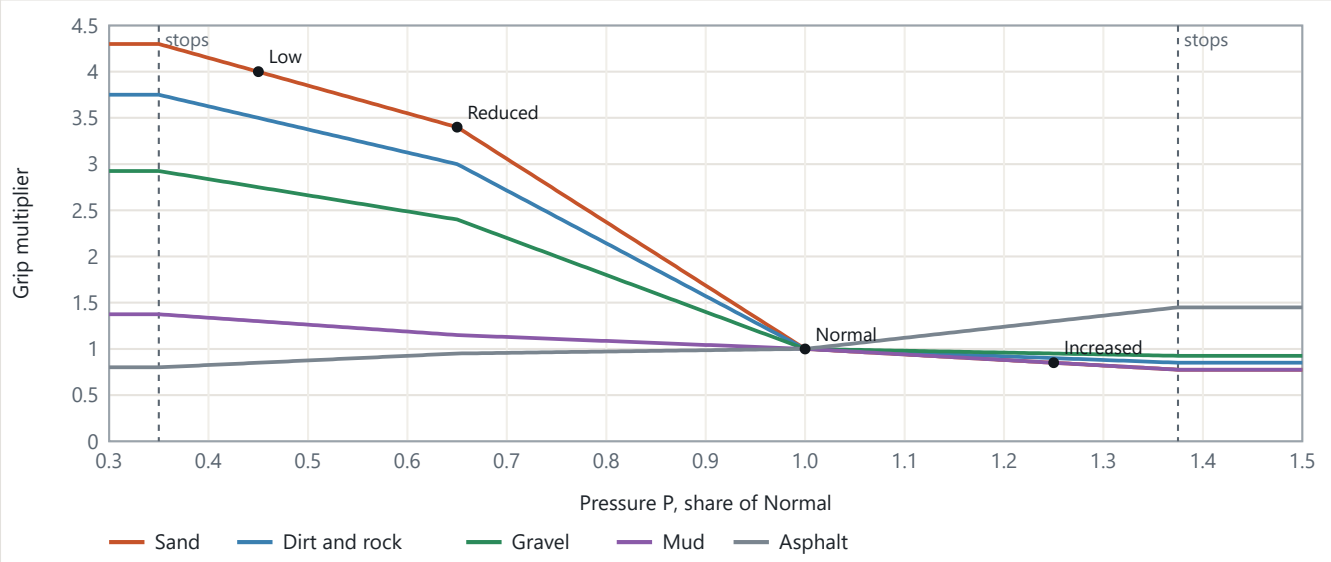


Figure 4. The grip multipliers along the pressure scale, default numbers. Straight between the modes, half a step on past either end, then flat.

The segment above Normal always uses the stored Increased numbers, also when Increased is switched off (as in the insanity mode). In hot weather Normal's pressure rises above 1 and slides a little toward them: at 35 °C, 39% of the way, which is asphalt ×1.12, dirt ×0.96 and fuel ×0.94 with the default numbers, before the rubber factors.

For the sounds and the air tank the pressure is also counted in modes, unclamped (`PressureInModes`):

$$m(P) = i + (P - P_i) \,/\, (P_{i+1} - P_i) \tag{4}$$

27. How fast the pressure moves

The pressure moves at one pace whatever the step: `Seconds` is the time from Normal to Low, a span of 0.55 on the scale (`PressureStep`, `tire_pressure.cpp:847-852`). Each pass moves it by

$$\Delta P = 0.55 \cdot \Delta t \,/\, S,\ \Delta t = 0.05\ \text{s} \tag{5}$$

and snaps it onto the target once it is within one step. $S = 0$ changes it at once. A change from P_a to P_b therefore takes $|P_b - P_a| \,/\, 0.55 \cdot S$:

Table 13. How long a change takes

Between	Default (6 s)	Hard mode (30 s)	Insanity mode (25 s)
Low and Reduced	2.2 s	10.9 s	9.1 s
Low and Normal	6.0 s	30.0 s	25.0 s
Low and Increased	8.7 s	43.6 s	no Increased
Reduced and Normal	3.8 s	19.1 s	15.9 s
Reduced and Increased	6.5 s	32.7 s	no Increased
Normal and Increased	2.7 s	13.6 s	no Increased

The choice itself counts at once; only the pressure follows. The fixed Δt makes the real time a little longer than the table, by the time each pass takes beyond its 50 ms sleep.

Standing still

With `StandStill` on, the pressure only moves while the truck stands:

$$\text{standing} = (\text{speed unknown}) \text{ or } |v \cdot f| < 0.3\ \text{m/s} \tag{6}$$

v is the chassis body's velocity and **f** its forward axis (`tire_pressure.cpp:908-916, 1803`); 0.3 m/s is about 1 km/h. A paused game leaves its last velocity in the body, so a truck that moved less than 0.01 mm in each of 4 passes (0.2 s) counts as still, whatever the velocity says. A change that is under way pauses where it is when the truck moves and goes on when it stops. The test uses the previous pass's speed, one pass (50 ms) late.

The air tank

The air for a filling comes from a tank, and only the tank's compressor makes noise (`tire_pressure.cpp:860-879`). Raising the pressure by Δm modes takes $0.17 \cdot \Delta m$ of the tank. Below 0.45 the compressor starts, and fills 0.04 of the tank a second until it is full. From full, that is the second filling from Low to Normal or the fourth filling of one mode; the compressor then runs for about 17 seconds.

28. Grip: what the game has and what the mod writes

SnowRunner keeps three grip values that matter here:

- each wheel's asphalt grip, `BodyFrictionAsphalt` in the tire files (wheel +0xA4), used on paved ground;
- each wheel's mud grip, `SubstanceFriction` (wheel +0xA0), used where the wheel is in mud;
- the truck's ground grip list, one value per wheel (truck action +0x98), filled from the tire's `BodyFriction` once the truck moves, and used on every other ground.

The game marks each frame which ground a wheel stands on, with a byte per kind where that layer is more than half of the mix (wheel +0x1C8 paved, +0x1C9 gravel, +0x1CA sand, +0x1CD hard). The mod picks the factor for the ground list in that order of priority: sand, then gravel, then rock (hard), otherwise dirt (`GroundFactor`, `tire_pressure.cpp:898-903`). On paved ground the game uses the wheel's asphalt value through its contact function (Steam RVA 0xc402d0) instead.

With $g(P)$ the factors read off the scale, r_{soft} and r_{hard} the rubber factors of the outside temperature, k the wheel's vanilla balance factor and F the asphalt floor, each wheel gets (`tire_pressure.cpp:1744-1782`):

$$\mathrm{asphalt} = \mathrm{A_0} \cdot \mathrm{g_{asphalt}}(\mathrm{P_{air}}) \cdot \mathrm{r_{hard}} \cdot \max(\mathrm{k}, \mathrm{F} \,/\, \mathrm{A_0}) \tag{7}$$

$$\mathrm{mud} = \mathrm{M_0} \cdot \mathrm{g_{mud}}(\mathrm{P_{air}}) \cdot \mathrm{r_{soft}} \cdot \mathrm{k} \tag{8}$$

$$\mathrm{ground} = \min(\,10, \mathrm{G_0} \cdot \mathrm{g_{ground}}(\mathrm{P_{air}}) \cdot \mathrm{r} \cdot \mathrm{k}\,) \tag{9}$$

A_0, M_0 and G_0 are the tire's own values. g_{ground} is the dirt, gravel, sand or rock factor by the ground; r is r_{hard} for rock and r_{soft} for the others. The max with F/A_0 only applies with a floor set. A worn out tire gets 0 in all three.

The body's own friction (body +0xCC) is never written. Writing it once gave frictions in the thousands, flat tires and a rolled truck (2026-10-02, `tire_pressure.cpp:661-663`).

Table 14. The default numbers (panel.h:131-134)

Value (ini key)	Low	Reduced	Normal	Increased
Dirt and plain ground OnModelFriction	×3.5	×3	×1	×0.9
Gravel GravelFriction	×2.75	×2.4	×1	×0.95
Sand SandFriction	×4	×3.4	×1	×0.85
Rock RockFriction	×3.5	×3	×1	×0.9
Asphalt (paved) BodyFrictionAsphalt	×0.85	×0.95	×1	×1.3
Mud SubstanceFriction	×1.3	×1.15	×1	×0.85
Radius offset AdditionalRadiusOffset	10 cm	6 cm	none	none
Fuel use FuelConsumptionModifier	×1.5	×1.25	×1	×0.85
Steering speed SteeringSpeedModifier	×0.7	×0.85	×1	×1.1

Where the numbers come from (`panel.h:118-130`): Reduced and Low are the "default tire inflation" set of an Expeditions tire file, all but asphalt. Expeditions has 1.3 and 2.0 there for its rock models, but in SnowRunner that value only acts on paved ground, where an aired down tire loses grip to its softer sidewall, so the mod has 0.95 and 0.85. Increased is a road mode: Expeditions' newer tire files trade grip for fuel and steering, and the mod adds better asphalt grip. The split into gravel, sand and rock is the mod's own: sand gains the most from a long contact patch, rock as much as dirt, gravel the least.

Base grip (`BaseGround`, `BaseAsphalt`, `BaseMud`) multiplies all four modes, Normal too, before the scale's clamps (`tire_pressure.cpp:366-374`).

29. Flattening

An aired down tire in Expeditions is a smaller collision cylinder: the game takes the mode's `AdditionalRadiusOffset` off the radius, in metres (its RVA 0xcd40c0). SnowRunner has the same wheel, but its wheel shader only draws a tire so flat. Past that the tread is drawn under the ground: a scout on 0.47 m tires at 10 cm sank 37 mm (2026-10-04).

The game squashes the drawn tire by an amount it takes from the hub's height over the contact (the wheel model's update, Steam RVA 0xc235f0), and the shader (`SpinTires/TruckWheel.fxs`, `g_softParams.w`) pulls the tread in by less the further an angle a is from the contact. For a hub h below the radius R:

$$q = \min(1.5\ h,\ 0.25\ R) + 0.02,\ z = (\min(0.2\ R,\ q) + 0.1) \cdot 0.8\pi\ /\ R \tag{10}$$

$$\mathrm{pull}(a) = q \cdot (0.8 - 1.1 \cdot \min(1,\ a\ /\ z)) \tag{11}$$

On flat hard ground the tire's circle reaches under the ground by $R - (R - h) / \cos a$ at the angle a, out to where it leaves the ground, $a = \mathrm{acos}((R - h)/R)$. The drop is drawn flat as long as

$$R - (R - h)\ /\ \cos a \le \mathrm{pull}(a) + 0.005\ R \text{ for every such } a \tag{12}$$

Half a percent of R under the ground is let through: nobody sees it, and on very large wheels the shader's shape misses a flat contact by that much early on. `FlattenLimit` (`tire_pressure.cpp:569-584`) steps h by 1 mm and a by a quarter of a degree and keeps the last h that fits: 0.35 m: 7.1 cm, 0.47 m: 8.6 cm, 0.65 m: 10.2 cm, 0.8 m: 10.8 cm, 1 m: 10.3 cm.

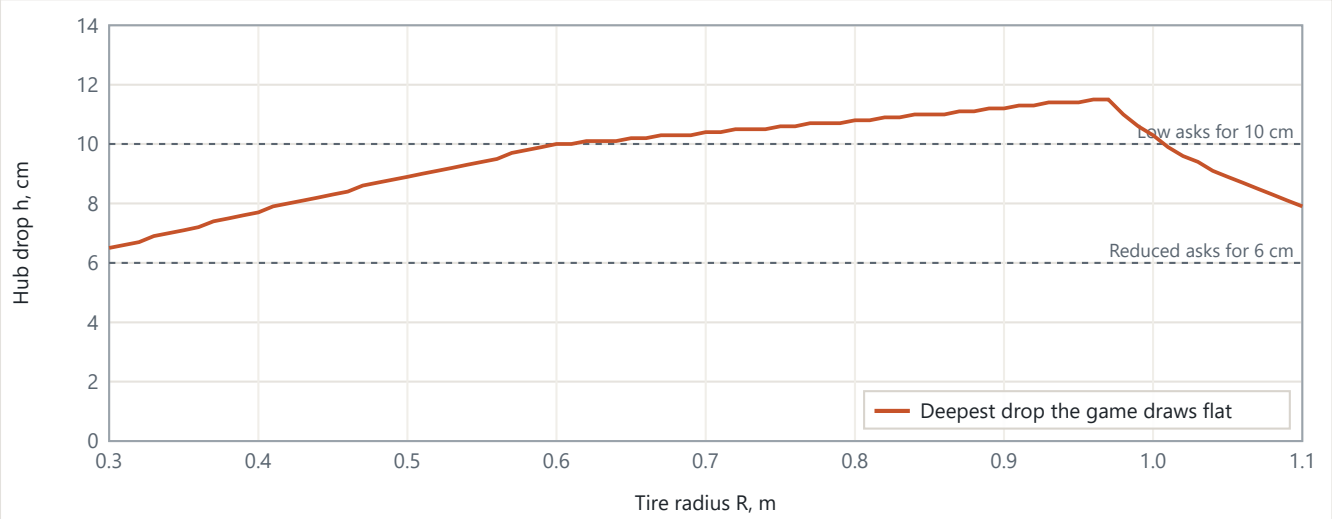


Figure 5. FlattenLimit: how far the hub can sit below the tire's radius before the drawn tread sinks into hard ground.

A tire file can already give the wheel an offset of its own (wheel +0xB4), which uses up part of that room. Every mode's offset then shrinks by the same share, so the deepest mode in use takes all the room there is (`Cylinder`, `tire_pressure.cpp:590-594`):

$$\mathrm{room} = \max(0,\ \mathrm{FlattenLimit}(R) - o_{\mathrm{own}} - \mathrm{sink}),\ \sigma = \min(1,\ \mathrm{room}\ /\ o_{\mathrm{deepest}}) \tag{13}$$

$$\mathrm{cylinder} = \max(\ 0.4\ R,\ R - o_{\mathrm{own}} - \min(o(P) \cdot \sigma,\ \mathrm{room})\ ) \tag{14}$$

Example, the test scout (R = 0.47 m, own offset 1.88 cm): FlattenLimit 8.6 cm, room 6.72 cm. Low gets 6.7 cm and Reduced 4.0 cm. The cylinder never goes below 40% of the radius, whatever the ini says. No offset goes past the room: in the cold Low's offset is carried past the deepest mode's (equation 3, up to 12 cm), and before 1.3.0 that went up to 1.2 times the room.

Tires that come soft

The game draws the squash from the distance d between the hub and the contact, not from the cylinder. Its wheel model update reads the contact at wheel model +0x21C and +0x220 and works out (Steam build, RVA 0xc239cc):

$$d = \sqrt{(x^2 + y^2)},\ \mathrm{squash} = \min(\ (R - d) \cdot 1.5,\ 0.25\ R\ ) + 0.02,\ \times \text{ wheel } +0x128 \tag{15}$$

On hard ground a firm tire touches at its cylinder, $d = c$, and the hub drop is $R - c$, the tire's own offset plus the mode's: what FlattenLimit counts. But each wheel carries a Havok soft contact built with its `SoftForceScale` (wheel +0x120), and a soft tire's contact sits inside the cylinder: the hub is lower again by sink $= c - d$, and the tread is drawn that much deeper. A modded tire that came soft went far too flat at Low (a tester, 2026-10-18). Since 1.3.0 the mod measures it under every wheel on the ground (wheel +0x1A4), each pass:

$$\text{sink} \leftarrow \text{sink} + 0.025 \cdot (\max(0, c_{\text{written}} - d) - \text{sink}), \text{ taken only for } 0.85\,c \le d \le 1.02\,c \tag{16}$$

0.025 a pass settles in about two seconds at 20 passes a second. A reading outside the window (a wheel in the air, a field a game update moved) is not taken. The sink is used in 5 mm steps, moved to the nearest step once it is three quarters of a step away, so the cylinder is not written again for every bump. It comes off the room first, equation (13), so a soft tire is flattened less and never drawn into the ground. A vanilla tire sinks next to nothing: the scout's test at 10 cm sank its tread 37 mm under the ground, which is exactly what equations (10) to (12) give with no sink. The log's wheel lines and the tab's truck section show each wheel's sink.

30. How a flattened tire rolls

A real tire's belt keeps its length. Flattened by h, its contact patch reaches $R \sin p$ to either side of the hub while the belt runs $R\,p$ over the same arc, with

$$\cos p = (R - h) / R \tag{17}$$

so the tire rolls as a wheel of radius

$$r_{\text{roll}} = R \sin p / p \tag{18}$$

which stays close to R for small drops. The game's wheel is its collision cylinder, $R - h$, and loses about three times as much speed per turn. With `RollingRadius=1` the mod raises the truck's gear speeds by the difference (`RollFactor`, `tire_pressure.cpp:601-609`):

$$\text{roll} = r_{\text{roll}}(\text{cylinder}) / r_{\text{roll}}(\text{stock}) \cdot \text{stock} / \text{cylinder} \tag{19}$$

p is held to $\cos p \ge 0.3$, and below $p = 0.01$ the radius is taken as R. The truck's gear speed list (truck action +0x58, at most 32 values) is raised so each gear's top grows by the mean roll of its wheels, worn out tires left out. The game governs a gear to $2a + 5$ rad/s for its number a, $a + 2$ in the high gear (the Gauges mod's reading of the game, its `models.h:127-135`), so each number gets:

$$a' = \text{roll} \cdot a + 2.5\,(\text{roll} - 1), \text{ high gear: } a' = \text{roll} \cdot a + 2\,(\text{roll} - 1) \tag{20}$$

Then $2a' + 5 = \text{roll} \cdot (2a + 5)$ exactly. Before 1.3.0 the number alone was scaled, which fell short most in the low gears: with $a = 5$, ×1.078 for ×1.117. For the scout: Low ×1.117, Reduced ×1.066; at Low the truck keeps 95.1% of its speed against 85.1% for the smaller wheel alone.

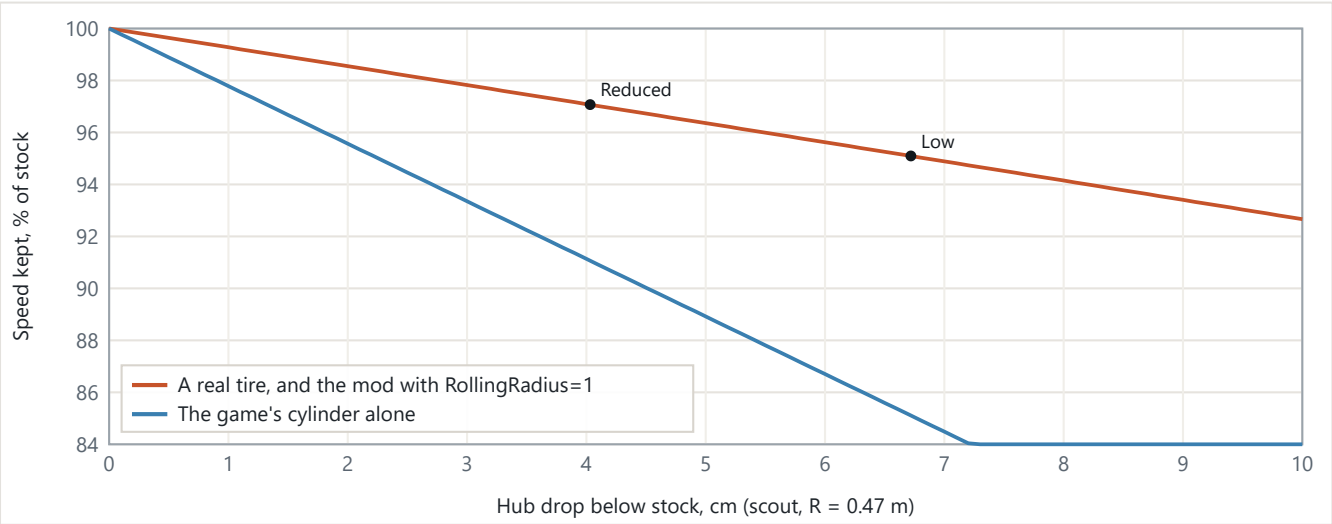


Figure 6. Speed per wheel turn as the tire flattens.

> **The pull.** If the game drove a wheel with a fixed torque at its axle, the push at the ground would be that torque over the cylinder's radius, and a smaller cylinder a shorter lever: for the scout at Low ×1.175, where a real tire pushes ×1.052. The Gauges mod's tests found that the game pulls less in a gear with a higher top speed (Gauges `models.h:1081-1086`), as a wheel torque that goes with 1 over the gear's top would. Then raising each gear's top by roll also takes the lever's extra back, and Low comes to ×1.052, a real tire's. With `RollingRadius=0`, as in the insanity mode, the lever's ×1.18 stands. None of this is measured; the Gauges mod's pull test, run at Normal and at Low on the same road, would settle it.

31. Fuel and steering

The game's fuel rate (Steam RVA 0xd670b0) multiplies by the sum of two values the truck carries: the engine's and the gearbox's `FuelConsumption` (vehicle +0xE38 and +0xE3C). The mod writes each as its stock value × the mode's fuel factor, but only while the truck rolls; standing, the factor is 1. Rolling starts above 0.5 m/s and lasts while above 0.25 m/s, as Expeditions applies its modifier only above a speed (its fuel rate, RVA 0xe168a0).

The steering speed is the truck's `SteerSpeed` (vehicle +0x69C) × the mode's factor; the game reads it every frame.

Every value the mod writes follows one rule (`ApplyValue`, `tire_pressure.cpp:745-757`): a value that differs from what the mod wrote last is taken as the new stock, if it lies in a range a tire or truck file can hold (fuel 0 to 1000, steering 0.0001 to 10, ground grip 0.05 to 10, gear speeds −1000 to 1000); the mod then writes min(cap, stock × factor). So a truck changed in the garage, or a value another mod writes, becomes the new stock instead of being overwritten.

32. Rolling drag

With `RollingDrag=1` (or hard or insanity mode) the mod writes each grounded, unbroken wheel's Havok angular damping (body +0x216, a half float: a float's upper 16 bits, rounded up by 1.003906 first). Havok slows a body's spin by $c \cdot \omega$, a torque of $I \cdot c \cdot \omega$, and the tire's grip passes it to the truck. A rolling resistance is the force $Crr \cdot N$, the torque $Crr \cdot N \cdot r$. So (`rolling_drag.h`):

$$c = c_{stock} + Crr \cdot f(p) \cdot N \cdot r \,/\, (I \cdot \max(|\omega|, 2\ \mathrm{m/s} \,/\, r)),\ f(p) = (p \,/\, p_{Normal})^{-0.5},\ N = M\ g \,/\, n$$

c_{stock} is wheel +0xAC, the game's own figure, never the body's word read back. r is wheel +0x94. I is the wheel body's inertia about its axle: 1 over body +0x220's x, y or z, the one that stands apart (`RollAxle`). ω is the body's angular velocity (+0x240) along that axle's column of the rotation (+0x170). M is the mass the physics moves: model +0x784, the game's own sum of the truck's bodies but the wheels (the chassis, a cab on a hinge, an antenna, the steering pivot; the chassis's body alone misses the cab), and every wheel body's 1/w (body +0x22C). In N each body comes with its Havok gravity factor: N = (model +0x784 × the chassis's factor + Σ each wheel's 1/w × its own) × g / n. Havok keeps the factor as a half in the body's motion after the velocities, at body +0x282. The game's code confirms the place: the truck file's GravityFactor goes into the body's construction info, and Havok's body constructor stores it there beside the damping, the same in both builds read. The truck's line gives the chassis's factor. n is the wheels with contact (+0x1A4). p is the mod's pressure share as it moves. c is held at 20 at most (the probe measured up to 8; at 60 frames a second c·dt reaches 1 at c = 60); the line a second counts the frames where it bites.

ω is the body's own spin, not v / r, so a wheel spinning in sand gets the same torque as one rolling. Above the floor the torque is $Crr \cdot N \cdot r$. Below it the torque falls with the spin, to nothing standing: the damping never pushes a wheel. Over all the wheels the drag slows the truck by $Crr \cdot f \cdot g$, whatever its mass.

Table 15. Crr by the ground flags the game's check leaves in the wheel

Ground	Flag	Crr	At Normal, m/s²	At Low, m/s²
Paved	`+0x1C8`	0.010	0.10	0.15
Rock	`+0x1CD`	0.015	0.15	0.22
Gravel	`+0x1C9`	0.020	0.20	0.29
Dirt, mud and no flag	none	0.030	0.29	0.44
Sand	`+0x1CA`	0.060	0.59	0.88
Ice	`+0x1C0`	0	0	0

Where several flags are set: ice, then paved, sand, gravel, rock, and dirt for the rest. No flag counts as dirt. Mud sets none, so mud gets dirt's 0.030 on top of the game's own mud drag; +0x1CE is not mud (it is on over paved road). Ice is 0.

When it writes. `RollingDragFrame` runs in the truck update, on the game's thread, every frame, on reads of that frame. A wheel's *c* goes in when the mod has no word of its own in it, when the body holds another word than the mod's last (the game wrote it: compared as a word, never taken as a stock), or when *c* moved by over 2 % since. The mod keeps its last word itself. A broken wheel (model +0x328) keeps the game's own. A wheel off the ground, the truck left for another and the setting off get the stock back, where the body still holds the mod's word. While the probe (`ProbeRoll`) holds a damping of its own, the drag waits. Without the truck update hook it stays off, with a log line.

The log. A line per truck with the chassis body's 1/w, the files' sum of the bodies (model +0x784), the wheels, the gravity factor, *N* and the mass, the first wheel's *I*, *r* and stock damping, the Crr and the pressure factor. With `ProbeRoll=1` also one a second while the truck moves: each wheel's ground, Crr × *f*, the *c* in its body, its spin, and the torque $I \cdot (c - c_{stock}) \cdot \omega$ against $Crr \cdot f \cdot N \cdot r$.

The probe's coast-downs showed that body +0x216 reaches the truck as $I \cdot c \cdot \omega$, for torques well above any rolling drag writes. Fuel: the fuel factors of hard mode and the insanity mode are 1, as the engine burns more against the drag by itself. The default modes keep theirs until a drive says otherwise.

33. Tire damage

A soft tire run too fast flexes and heats. Each mode below Normal has a speed above which its tires take damage, in ticks. With *v* the forward speed and r_{speed} the temperature's factor (see the outside temperature):

$$\text{too fast} = v > v_{min} \cdot r_{speed} \tag{21}$$

$$s = \text{clamp}(\,(v - v_{min}) \,/\, (v_{max} - v_{min}),\, 0,\, 1\,),\ \text{damage per tick} = \lfloor\, D_{min}\,(1 - s) + D_{max}\, s\,\rfloor \tag{22}$$

with v_{min} and v_{max} both × r_{speed} (Expeditions' formula, its RVA 0xccfdf0). While too fast a timer runs on the measured time; when it passes the tick length, every wheel on the ground takes the damage and the timer starts again. Below the speed for a full second the timer resets: Expeditions checks every frame, the mod 20 times a second, and a single missed reading must not cost a whole tick (`tire_pressure.cpp:1839-1883`).

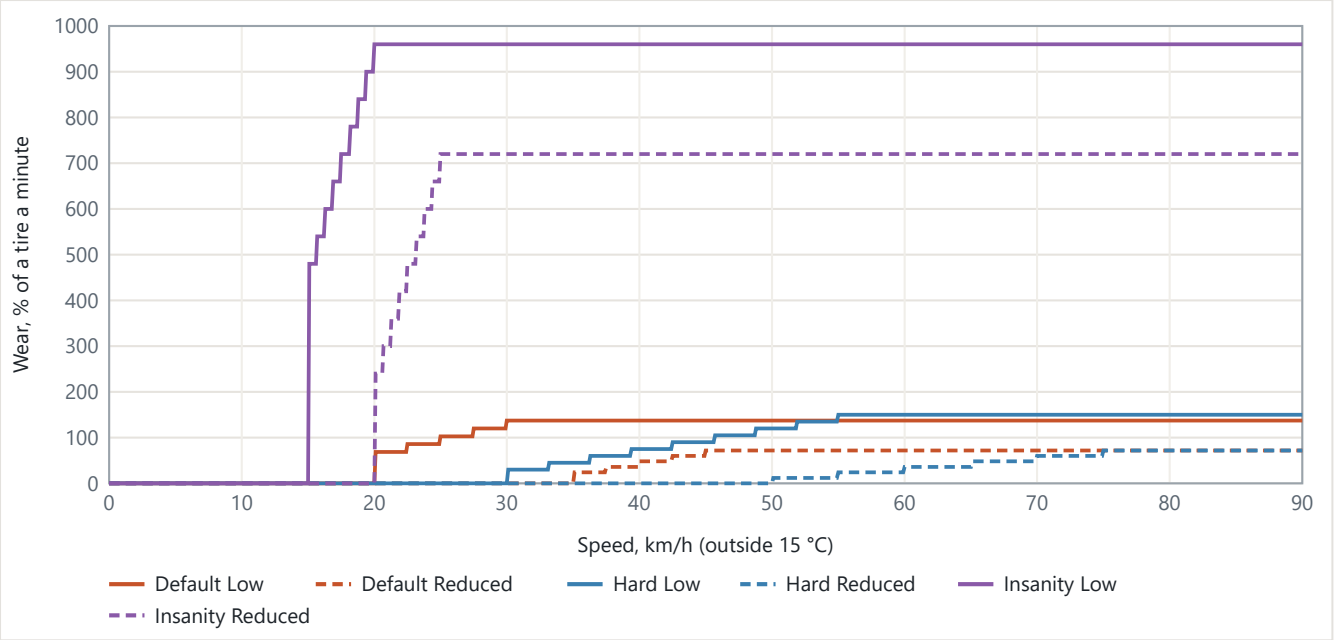


Figure 7. How fast a tire wears by speed, for a SnowRunner wheel of 50 damage points. 100% a minute wears a new tire out in a minute.

Table 16. Tire damage, and how long a new tire lasts (50 damage points, 15 °C)

Set and mode	Speeds	Damage per tick	Tick	New to worn out, just too fast	At full speed
Default Low	20 to 30 km/h	4 to 8	7 s	91 s	49 s
Default Reduced	35 to 45 km/h	2 to 6	10 s	250 s	90 s
Hard Low	30 to 55 km/h	2 to 10	8 s	200 s	40 s
Hard Reduced	50 to 75 km/h	1 to 6	10 s	500 s	90 s
Insanity Low	15 to 20 km/h	8 to 16	2 s	14 s	8 s
Insanity Reduced	20 to 25 km/h	4 to 12	2 s	26 s	10 s

Where the numbers come from (`panel.h:126-129`): Expeditions' off-road tires take 2 to 4 at Low from 27 to 36 km/h and 1 to 3 at Reduced from 36 to 54 km/h (`_templates\trucks.xml`). The mod doubles the damage and lowers the speeds, set by driving on 2026-10-04, as a SnowRunner truck is slower off the road. Version 1.0.0 used 15 to 25 and 25 to 35 km/h, which more driving found too low. The tick lengths have no stated source.

How damage reaches the game

The truck's damage lives in blocks behind a handle at vehicle +0x218 (the game's accessor, Steam RVA 0xa749a0): the block's address is the handle with its second byte cleared (mask `0xFF00FFFFFFFFFFFF`); its 16-bit flags at +0x10 must have bit 0 set and bit 2 clear; its data starts at +0x38. The wheels' list holds a pointer at +8 and a count at +0x10, and each wheel's block is three values: damage (int), capacity (int) and effect (float). The mod only touches a block whose capacity equals the one the wheel model was built with (wheel model +0x324) and whose damage is between 0 and that capacity, and adds to each wheel on the ground (wheel +0x1A4):

$$\text{damage}' = \min(\ \text{capacity},\ \text{damage} + \text{added}\ ) \tag{23}$$

Then, on the game's own thread, it runs the game's damage update for the truck (Steam RVA 0xd6f690). That update is what flattens a worn out tire, mends a repaired one, sets each part's effect, sums the truck's damage (vehicle +0x108 and +0x10C) and tells the truck's script, which brings up the truck card and turns the wheel icon red.

Without the game-thread path the mod breaks a worn out wheel by hand: it writes the wheel's broken radius (wheel +0x9C) into its radius (+0x94) and its shape's radius (shape +0x28), the broken damping into the body as Havok's 16-bit float (the upper half of the 32-bit float of $v \times 1.003906$, the game's rounding), and the broken flag (wheel model +0x328). The game's own break builds a new Havok shape, which needs its own thread; the by-hand break cannot, and the game's display only follows at the next damage the game deals itself.

A worn out tire

When the wheel model's broken flag (+0x328) is set, the mod writes 0 into the wheel's asphalt and mud grip and its ground grip entry, and draws it flat (`ApplyWornWheel`, `tire_pressure.cpp:618-655`):

$$\text{cylinder} = \max(\, 0.4\ \text{b}, \text{b} - \text{FlattenLimit}(\text{b}) \,), \text{b} = \text{the broken radius} \tag{24}$$

A worn wheel is left out of the gear speeds' mean. A repair makes the game rebuild the shape at stock size with the zero grip still on it; the mod sees a cylinder back near stock, forgets the worn state and writes the mode's values again.

34. The outside temperature

Tires are taken as set at 15 °C. The air in a tire of fixed volume follows the gas law on absolute pressure, gauge pressure plus the atmosphere's 14.7 psi. With Normal at 35 psi gauge the atmosphere is 14.7 / 35 = 0.42 of Normal's pressure, and a tire set to P at 15 °C holds, at T (`outside.h:109-124`):

$$\text{P}_{\text{air}} = (\text{P} + 0.42) \cdot (\text{T} + 273.15) \,/\, 288.15 - 0.42 \tag{25}$$

At −20 °C Normal holds 0.83 and Low 0.34 (76% of its own); at 35 °C Normal holds 1.10. A low pressure loses a larger share of itself, because the atmosphere's part does not shrink. The mode factors are read off the scale at P_{air}, so cold air moves every mode a little lower on the scale.

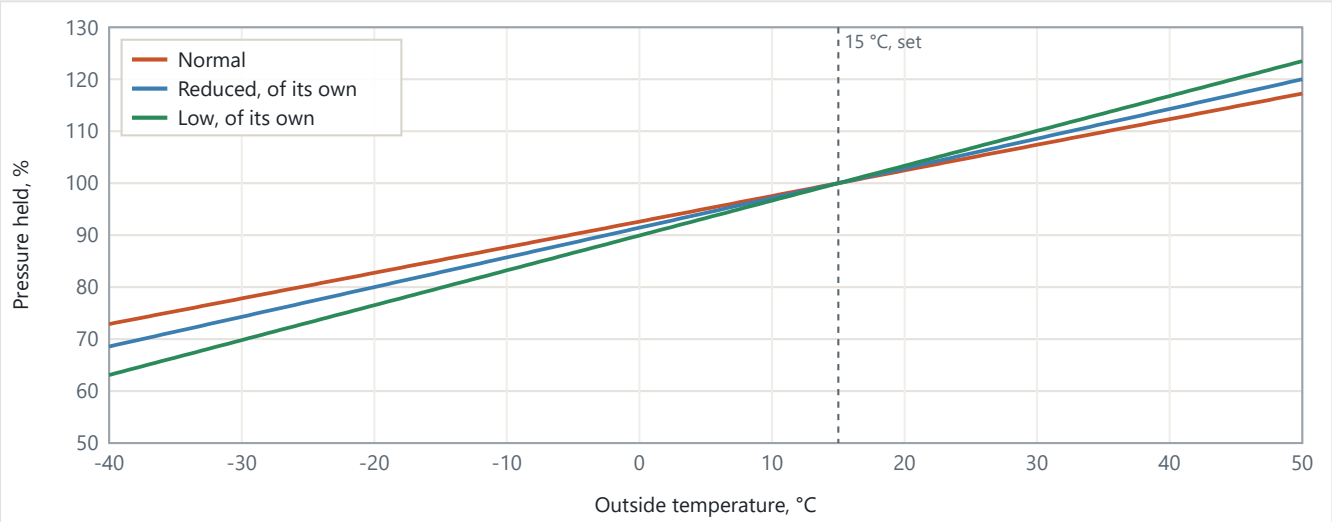


Figure 8. The gas law: the pressure each mode holds, as a share of what it holds at 15 °C. A low pressure loses more of itself in the cold.

Rubber also changes with the temperature: cold rubber hardens and grips less, most on hard surfaces. With below = max(0, 15 − T) and above = max(0, T − 15) (`outside.h:112-121`):

$$\text{r}_{\text{hard}} = \text{clamp}(\, 1 - 0.006 \cdot \text{below} + 0.003 \cdot \text{above}, 0.7, 1.08 \,) \tag{26}$$

$$\text{r}_{\text{soft}} = \max(\, 0.85, 1 - 0.003 \cdot \text{below} \,) \tag{27}$$

$$\text{r}_{\text{speed}} = \text{clamp}(\, 1 + 0.006 \cdot (15 - \text{T}), 0.75, 1.25 \,) \tag{28}$$

r_{hard} scales asphalt and rock, r_{soft} dirt, gravel, sand and mud. The damage speeds are multiplied by r_{speed}: a soft tire's sidewalls heat as they flex, so in cold air the damage starts at a higher speed and in hot air at a lower one. The clamps bite at −35 °C (hard and soft), −26.7 °C (speed) and 41.7 °C (hard). The code gives the reasoning (cold rubber "a fifth less" on hard ground at 20 below) but no source for the slopes.

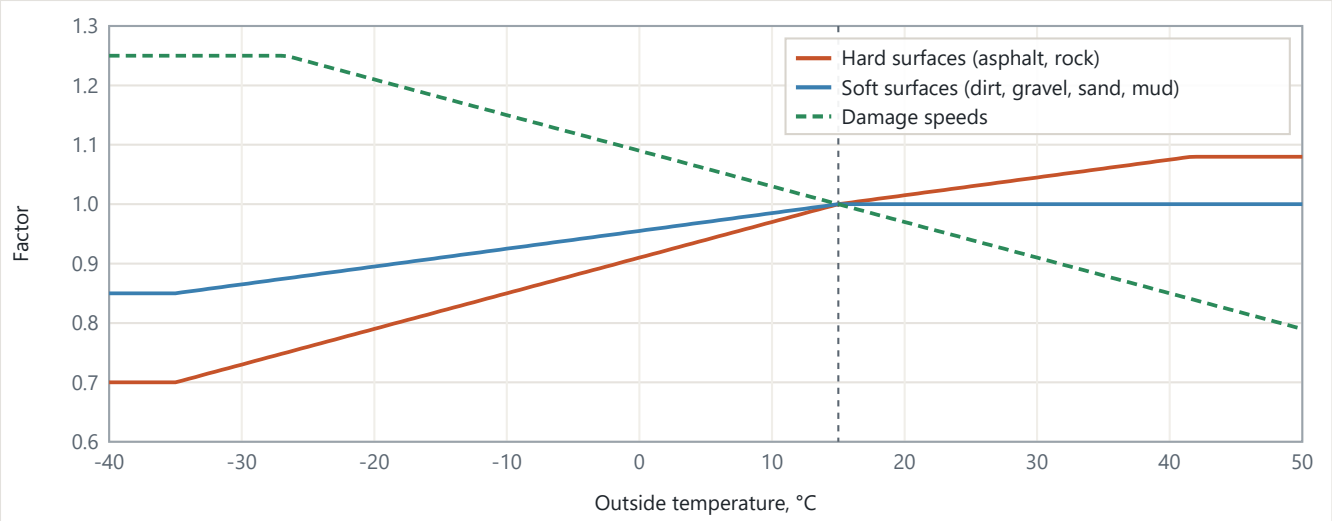


Figure 9. The rubber factors and the damage speed factor.

Table 17. Low pressure, default numbers, at different outside temperatures: everything together

Outside	P at Low	Dirt	Sand	Asphalt	Mud	Offset asked	Wear above
-30 °C	0.314	×3.24	×3.72	×0.58	×1.19	12.0 cm	25 km/h
-20 °C	0.344	×3.36	×3.85	×0.63	×1.23	12.0 cm	24 km/h
-10 °C	0.375	×3.41	×3.91	×0.69	×1.25	11.5 cm	23 km/h
0 °C	0.405	×3.45	×3.95	×0.75	×1.27	10.9 cm	22 km/h
15 °C	0.450	×3.5	×4	×0.85	×1.3	10.0 cm	20 km/h
30 °C	0.495	×3.39	×3.86	×0.91	×1.27	9.1 cm	18 km/h
45 °C	0.541	×3.27	×3.73	×0.97	×1.23	8.2 cm	16 km/h

The temperature is rounded to half a degree before any of this, so a drifting reading does not rewrite the wheels every pass. With `Temperature=0` every factor is 1.

Where the temperature comes from

From the Gauges mod's shared block (see Shared memory) when it is open and fresh: magic "GAUG", version 1 or later, size 64 or more, written within the last 2 seconds, and a temperature between −90 and 90 °C. Otherwise from `AmbientC` in the ini (−40 to 50, 15 by default). The block is looked for every 5 seconds until found, then kept open.

35. Hard mode and the insanity mode: the full numbers

Both replace the player's numbers and options while ticked; the ini keeps the player's own, and unticking brings them back (`PresetSettings`, `panel.h:197-201`). The insanity mode wins if both are set in the ini; the tab's two boxes exclude each other. The shared state flag "hard mode" is set for either.

Table 18. What each one forces (panel.h:156-194)

Setting	Hard mode	Insanity mode
Seconds (Normal to Low)	30	25
Base grip, all three	1	1
Vanilla balance	on, strength 1	on, strength 2
Asphalt floor	off	off
Increased	on	off
Tire damage	on	on
Rolling radius	on	off
Change only standing	on	on
Temperature	on	on
Rolling drag	on	on
Left alone	key, beep, volume, panel, pads, AmbientC	the same

Table 19. Hard mode (kHardModes, panel.h:148-151)

	Low	Reduced	Increased
Dirt	×1.6	×1.3	×0.92
Gravel	×1.3	×1.15	×0.95
Sand	×1.9	×1.45	×0.85
Rock	×1.6	×1.3	×0.92
Asphalt	×0.8	×0.9	×0.97
Mud	×1.35	×1.18	×0.9
Radius offset	5 cm	2.5 cm	none
Fuel	×1	×1	×1
Steering	×0.8	×0.9	×1.05
Wear from	30 km/h	50 km/h	no wear
Most wear from	55 km/h	75 km/h	no wear
Damage per tick	2 to 10	1 to 6	
Tick	8 s	10 s	

Hard mode's reasoning (`panel.h:142-147`): halving a tire's pressure lengthens its contact patch by about half, which is that much more grip on soft ground and a little less on hard; rolling resistance rises as the pressure falls, and the mod's rolling drag charges it, so the fuel factors are 1; a soft tire's sidewalls flex and heat at speed; a change takes half a minute, as a truck's onboard compressor does. The numbers are the mod's; the reasoning is the physics.

Table 20. Globalhacker insanity mode (kInsaneModes, panel.h:175-178)

	Low	Reduced
Dirt	×2	×1.5
Gravel	×1.6	×1.3
Sand	×2.3	×1.7
Rock	×2	×1.5
Asphalt	×0.4	×0.45
Mud	×1.5	×1.25
Radius offset	12 cm	6 cm
Fuel	×1	×1
Steering	×0.6	×0.75
Wear from	15 km/h	20 km/h
Most wear from	20 km/h	25 km/h
Damage per tick	8 to 16	4 to 12
Tick	2 s	2 s

The insanity mode is built from the player Globalhacker's own hardcore set (2026-10). With the balance at strength 2, a modded tire twice as strong as the best vanilla one ends at half of vanilla (see the next section); Low then takes 0.4 of that on asphalt. At 20 km/h Low wears a new tire out in 8 seconds.

36. The vanilla balance

A tools script (`tools/vanilla_tires.ps1`) reads every truck tire in the game's `initial.pak`, following each `WheelFriction` through its templates, and leaves out tires without a class and invisible ones. Of 441 tires with 94 different grip sets, 16 are not matched or beaten in all three of ground (`BodyFriction`), asphalt and mud by any other. Those 16 are the envelope: any vanilla tire is inside it.

Table 21. The 16 vanilla tires no other vanilla tire matches or beats in all three (vanilla_tires.h)

	Ground	Asphalt	Mud
1	0.9	3.2	0.4
2	1	3.2	0.2
3	1.5	1.5	2
4	1.6	1.5	1.9
5	2	3	1.2
6	2.2	2.6	1.3
7	2.8	1.2	1.8
8	2.85	1.2	1.78
9	3	0.5	8
10	3	0.8	3.5
11	3	1.2	1.3
12	3.1	0.8	3.4
13	3.1	1	1.9
14	3.2	0.8	1.5
15	3.2	1	1.3
16	3.3	0.6	2.2

For a tire with ground, asphalt and mud grip *G*, *A*, *M*, the factor is the smallest cut that brings some envelope tire level with it or ahead of it in all three (`vanilla_balance.h:9-22`):

$$f = \min(\,1,\ \max \text{ over the envelope of } \min(\,G_e/G,\ A_e/A,\ M_e/M\,)\,) \tag{29}$$

A ratio counts as very large when the tire's own value is 0. A vanilla tire has $f = 1$. The strength s (0 to 2) sets how much of the cut is applied (`vanilla_balance.h:27-32`):

$$k = 1 - s\,(1 - f) \text{ for } s \le 1,\quad k = f \cdot (1 - (s - 1)(1 - f)) \text{ for } s > 1 \tag{30}$$

At $s = 2$, $k = f^2$: a tire three times the vanilla grip gets $f = 1/3$ and $k = 1/9$, and ends at a third of vanilla. The same k multiplies the ground list, mud and asphalt; the modes and base grip act on top. Asphalt can be held up by the floor F: $\max(k, F/A_0)$, equation (7). The floor follows an idea by Naybour.

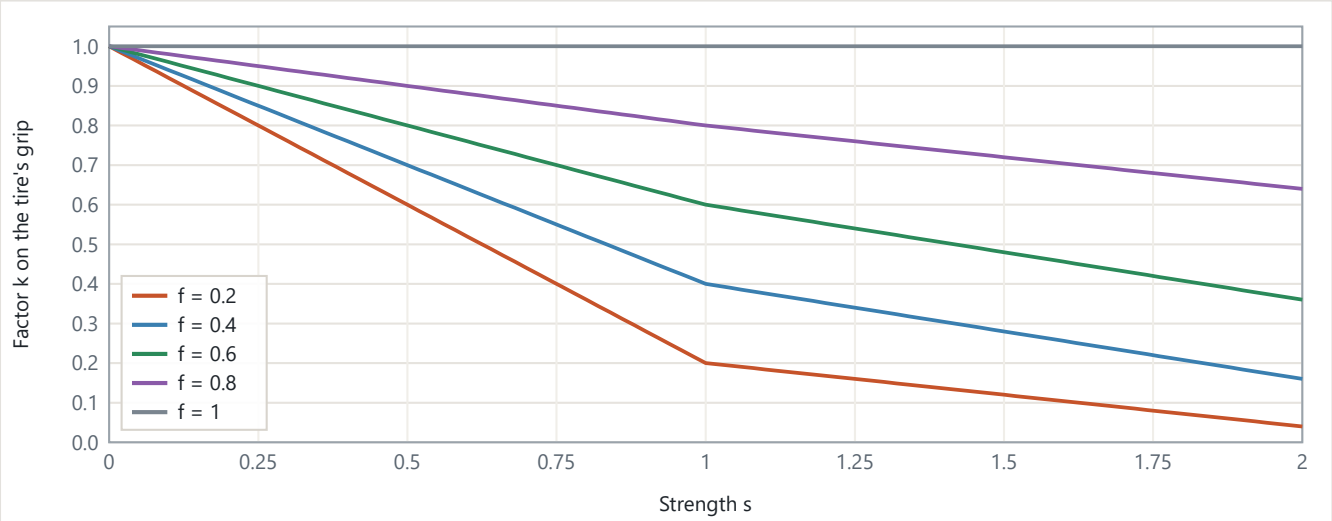


Figure 10. The balance factor by strength, for tires of different factor f. At 1 the tire ends level with vanilla, at 2 at f^2 of its own.

Inputs per wheel: the ground value is the truck's ground grip list for that wheel, asphalt and mud the wheel's own stock values. The list only fills once the truck moves, so the balance (and its readout on the tab) waits for the first

metres. The envelope is the whole game's: a modded highway tire is compared with the best vanilla tire of any kind, not of its class.

37. Getting into the game: the loader

`version.dll` is a proxy. Windows loads a DLL the exe imports from the exe's folder before System32, and SnowRunner imports `version.dll`. The proxy has the same 17 exports; each is a jump through a table filled at load (`loader.cpp`, `version_stubs.asm`):

- If `version_chain.dll` sits next to it (another loader, renamed), each export is taken from that one when it has it, otherwise from `System32\version.dll`. A copy of the proxy running as `version_chain.dll` forwards to System32 only and loads nothing, so two copies cannot call each other in a circle.
- `.asi` files in the folder are loaded on a thread of their own once the loader lock is free. Only names ending exactly in `.asi` load, so `x.asi_off` is skipped; a module another ASI loader has already loaded is left alone. It writes `AsiLoader.log`.
- If `System32\version.dll` cannot be loaded the proxy refuses to load, and the game reports the missing DLL instead of crashing later. An export that neither file has (Wine's and Proton's `version.dll` lack `GetFileVersionInfoByHandle`) fails only its own call, with `ERROR_PROC_NOT_FOUND`, and `AsiLoader.log` names it.

The ASI does nothing outside `SnowRunner.exe`. A mutex `Local\SnowRunnerTirePressure-<process id>` lets only one copy run; a second one logs it and stops.

38. Finding the game's code

The mod carries no addresses. At every start it copies the game's image out of its own process (`ReadProcessMemory` in 64 KB pieces, 4 KB pages where a piece fails, unreadable pages left as zeros) and searches it (`build_find.h`). Patterns are only searched in the image's code sections; a place must match exactly once.

What it must find

- The wheel's virtual table, from the game's type information for `.?AUTRUCK_WHEEL_PARAMS@combine@@`, and the cylinder shape's, from `.?AVhkpCylinderShape@@`. From a type name the type descriptor is 16 bytes before it; the object locator is the one with signature 1 and offset 0 that points at that descriptor and at itself; the table starts 8 bytes after the pointer to the locator. Exactly one must be found.
- The control global, which leads to the truck you drive (+8). Found three ways, the first that matches exactly once wins: (1) three accessor calls in a row with the Steam build's stack offsets, inside the truck update, where the second call's target is `lea rax,[rip+x]; ret` and *x* is the global; (2) the same with the stack offsets left open; (3) a short accessor between padding, `c3 cc×8 48 8d 05 ?? ?? ?? ?? c3`, that returns the object the global sits in, +0x18. The Game Pass build of October 2026 (stamp 6a607d39) had neither the call nor way 1, from a player's log of 2026-10-15; ways 2 and 3 came with 1.2.1.
- Six pieces of the game's code that use the same field offsets the mod writes. If a game update moves a field, its piece no longer matches and the mod stands down instead of writing to the wrong place.

Table 22. The layout checks (build_find.h:237-244)

Field checked	Bytes
SteerSpeed, vehicle +0x69C	`f3 0f 10 96 9c 06 00 00 f3 41 0f 5f d1`
Fuel, +0xE38 and +0xE3C	`f3 0f 10 83 3c 0e 00 00 f3 0f 58 83 38 0e 00 00 f3 0f 59 f0`
Gear speeds +0x58 by gear +0x74	`48 63 4b 74 44 0f 29 44 24 50`
Ground list, paved flag wheel +0x1C8	`f3 42 0f 10 0c b0 80 bf c8 01 00 00 00 0f 84`
Asphalt grip, wheel +0xA4	`f3 0f 59 8f a4 00 00 00 f3 0f 59 15`
Radius and own offset, +0x94 and +0xB4	`f3 0f 10 9b 94 00 00 00 f3 0f 5c 9b b4 00 00 00`

The game-thread path (all or nothing)

The damage update's start (`48 8b c4 48 89 48 08 55 56 57 41 54 ...`); the `GetForegroundWindow` slot in the import table; and the place in the truck update that calls it through that slot, whose function must start with the expected 33-byte head (four arguments stored, eight registers pushed). From the head the mod works out where the update's first argument lies relative to the return address: locals + 8 × pushes + 16, which is 0x3C8 + 0x40 + 0x10 = 0x418 in the Steam build.

Waiting for the code

Steam's exe only makes its code readable as it starts, so the mod looks up to 120 times, half a second apart. It stops as soon as everything is found. If the essentials are found but the game-thread path is not, the same report must come back 10 looks in a row (5 seconds) before the mod goes on without it, so a look made while the exe is still unpacking cannot pass for the real answer. If the essentials are still missing after a minute, the mod stands down: it writes nothing and the tab says so.

Table 23. What the search finds in the Steam build (the offline test checks these)

Place	Steam build
Time stamp	0x6a607c05 (22 July 2026)
Image size	0x2e02000
Control global	0x2a8eb78
Wheel vtable	0x2258918
Cylinder vtable	0x23c2c08
Damage update	0xd6f690
GetForegroundWindow import slot	0x21be360
Truck update, return address of the call	0xa57535
First argument, from the return address	0x418

`DumpImage=1` writes the copied image to `TirePressure_image_<stamp>.bin` next to the exe after the search. A Game Pass install cannot be copied out of its folder; this copy, taken from memory, can be sent in for a build the mod does not know.

39. The memory map

Table 24. Every field the mod reads or writes (Steam build)

Object	Offset	What	The mod
control (global)	+0x8	the truck you drive	read
vehicle	+0x48	the game's id for the truck: a text in braces behind the pointer (as Next Gen Gameplay reads it)	read
vehicle	+0x60	model; its +0x230 is the chassis body (velocity +0x230, forward axis +0x170, position +0x1A0), its +0x588 the kind's name	read
vehicle	+0x68	truck action	read
vehicle	+0x108, +0x10C	damage sum and capacity	read
vehicle	+0x200, +0x208	wheel model array (at most 64)	read
vehicle	+0x218	damage handle	read
vehicle	+0x69C	SteerSpeed	written
vehicle	+0xE38, +0xE3C	engine and gearbox FuelConsumption	written
truck action	+0x58, +0x60	gear speed list	written
truck action	+0x74	gear index	checked
truck action	+0x98, +0xA0	ground grip list	written
wheel model	+0x2C8	the wheel	read
wheel model	+0x21C, +0x220	the contact, from the hub (the sink)	read
wheel model	+0x324	damage capacity it was built with	read
wheel model	+0x328, +0x329	broken flag, old flag	read; written by the by-hand break
wheel	+0x18	body	read
wheel	+0x94	radius (the broken one once broken)	read; written by the by-hand break
wheel	+0x98, +0x9C	own radius, broken radius	read
wheel	+0xA0, +0xA4	mud grip, asphalt grip	written
wheel	+0xA8, +0xAC	broken and normal damping	read
wheel	+0xB4	own radius offset	read
wheel	+0x1A4	touches the ground	read
wheel	+0x1C8 to +0x1CD	ground kind flags: paved, gravel, sand, hard	read
body	+0x20	shape (hkpCylinderShape); its +0x28 is the radius	written
body	+0xCC	BodyFriction	never written
body	+0x216	damping, Havok 16-bit float	written by the by-hand break

Guards

- Every read and write goes through `ReadProcessMemory` and `WriteProcessMemory` on the game's own process, and addresses below 0x10000 are refused. An object the game frees under the mod is an error return, not a crash. No game code is changed, apart from the one import slot of the game-thread path.
- Wheel and cylinder are recognised by their virtual tables at every lookup, and the whole chain from the control global down is checked again before writes.
- A wheel's values are taken as stock only from a fresh-looking wheel: radius 0.05 to 5 m, own offset at most 0.3 *R*, asphalt above 0 and at most 10.5, mud 0 to 10.5, cylinder equal to *R* minus its offset. Anything else is left alone and logged (up to 8 times).
- Each value is recorded as soon as it is written, so a write that fails part way leaves a true record. Two floats count as equal within 0.0005 × max(1, |*b*|).
- List lengths are checked (ground list one value per wheel, gear list at most 32), a speed must be under 200 m/s, and floats from the ini must be finite.

40. The game-thread hook

The game's damage update builds Havok shapes and must run on the game's thread. The mod gets there without patching code: the truck update calls `GetForegroundWindow` through the import table once a frame, and the mod points that one slot at its own function (`HookTruckUpdate`, `tire_pressure.cpp:1060-1115`).

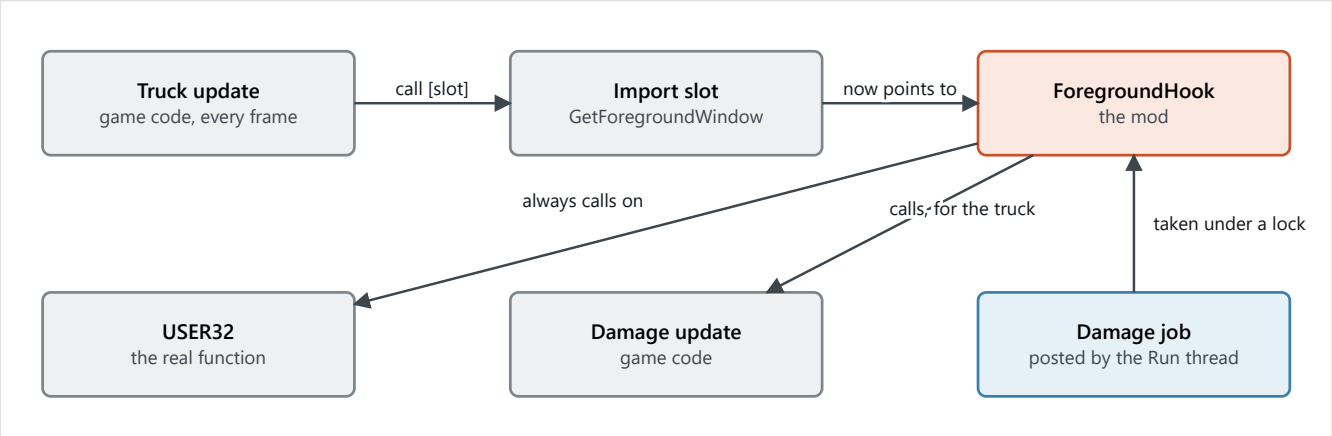


Figure 11. The game-thread hook. Only a call coming back to the truck update does anything; every call gets the real function's answer.

- Before the swap it checks the call (`FF 15` 6 bytes before the return address, pointing at the slot), the damage update's first 8 bytes, and that the slot's target is committed executable memory. The swap itself is one atomic pointer exchange.
- When the return address is the truck update's, the hook reads the update's first argument at the return address + 0x418, and the vehicle at its +0x20. If a damage job is posted for that same vehicle, which is the one you drive, it deals the damage and calls the damage update. It then calls the real `GetForegroundWindow` and returns its answer.
- The Run thread uses this path while the hook is in place and a truck update was seen in the last 500 ms; a job not taken within a second is dropped. Otherwise it falls back to the by-hand break.

41. Drawing

The panel, the warnings and the Tire Inflation tab are drawn by Next Gen's overlay, the same code as Next Gen Gameplay's (`ng_overlay.cpp`), with Dear ImGui on the game's device. It hooks Present on the game's own swap chains through method tables of their own: where ReShade is installed, ReShade's presents hand it each swap chain the game makes (`ng_reshade.cpp`); without ReShade it takes the game's imports of `CreateDXGIFactory1` and `D3D11CreateDevice` and gives each factory and swap chain they bring a method table of its own. DirectX's own tables are never written, so the Steam overlay, which hooks what those hold, never meets the overlay's

Present. The release brings ReShade 6.8.0 without its start banner, with a `ReShade.ini` that loads no effects and keeps Home for ReShade's own window (`tools/reshade_nobanner.py` makes that `dxgi.dll` from ReShade's own): on a tester's PC with RivaTuner, Discord's overlay and two monitors the game crashed without ReShade and not with it. Where ReShade is missing or does not take the overlay as an add-on, the start says so in the log, in a line at the top for 15 s that the first Next Gen mod in order draws, and in the menu above the tab's page (`NgReShadeCheck`). The NEXT GEN menu holds every Next Gen mod's tab in one window, on F2 or the key in `NextGen.ini`; while it is open the game gets no mouse and no keys. Next Gen Graphics offers its switches through `NgGraphicsGetApi` (`ng_graphics.h`): the first Next Gen mod to claim it in the block draws a Graphics tab from them, and Next Gen Graphics silences its F keys meanwhile. Up to 1.2 all of this was drawn through ReShade's add-on overlay.

42. The pad

With the overlay running, every 2 seconds the mod replaces each copy of the address of `XInputGetState` (from `xinput9_1_0.dll`) in the exe's data with its own filter (two copies in the Steam build: the input system and one filled lazily). The filter hides the panel's buttons from the game while they act on the panel: the open binding while held, the step chords, every panel button while the panel is open, and any of them still held after it closes until let go. It acts only while a truck is driven. Before the game has loaded XInput, the mod reads the pad itself through `xinput1_4.dll`; without the addresses found, the game also sees the presses. The same filter maps the two triggers through a table per trigger, built from `PadThrottleCurve` and `PadBrakeCurve` (the trigger's share to that power, 255 steps, identity at 1), only while a truck is driven.

43. Shared memory

The two mods talk through named blocks of shared memory, 64 bytes each, one per game process. Each is written about 20 times a second and read without locks: the writer makes the sequence odd, writes, and makes it even again; a reader copies the block and keeps the copy only if the sequence was even and did not change during the copy (up to 8 tries). A block whose tick is more than 2 seconds old means the writer is gone. Later versions only add fields at the end.

```
writer:  seq += 1 (odd)   barrier   fields   barrier   seq += 1 (even)
reader:  s = seq; if s odd, retry;   copy;   barrier;   if seq != s, retry
```

Table 25. Read: Local\SnowRunnerGauges.State.<process id>, written by the Gauges mod

Offset	Type	Field
0	u32	magic 0x47554147, "GAUG"
4	u32	version (1 or later accepted)
8	u32	size (64 or more accepted)
12	i32	sequence, odd while being written
16	u64	tick: GetTickCount64 at the write, ms
24	u64	vehicle (0: none)
32	f32	outside temperature, °C
36	f32	coolant, °C
40	f32	engine speed, rpm
44	f32	boost, bar
48	u32	flags: bit 0 vehicle, 1 temperature by map, 2 engine on
52	u32 × 3	reserved

Table 26. Written: Local\SnowRunnerTirePressure.State.<process id> (tire_state.h)

Offset	Type	Field
0	u32	magic 0x45524954, "TIRE"
4	u32	version 1
8	u32	size 64
12	i32	sequence
16	u64	tick, ms
24	u64	vehicle (0: no truck)
32	i32	mode: 0 Low, 1 Reduced, 2 Normal, 3 Increased
36	i32	modes in use, 3 or 4
40	f32	pressure, share of Normal's
44	f32	pressure after the temperature
48	f32	target: the mode's pressure
52	f32	outside temperature, °C
56	u32	flags, below
60	i32	wear: % of the most worn tire, −1 unknown

Table 27. The tire block's flags

Bit	Meaning
0	a truck is driven
1	the pressure is changing
2	held: the truck must stop for it to go on
3	too fast: the tires are taking damage
4	a tire is worn out
5	hard mode or the insanity mode
6	the outside temperature comes from the Gauges mod

The tire block is created once the game's code is found, so a mod that stood down publishes nothing. Before the first truck it is written twice a second. `tire_state.h` carries a reader's copy function for other mods.

44. Sounds

Table 28. The voices (sound.h:68-72)

Voice	Ini key	Source	Attack / release	Level	Plays
Air let out	AirOutSound	recording	0.005 / 0.12 s	0.56	loop while the pressure falls
Air going in	AirInSound	recording	0.005 / 0.12 s	0.44	loop while it rises
Compressor	CompressorSound	made by the mod	0.5 / 0.7 s	0.8	loop while it runs
Compressor stopping	CompressorStopSound	the game's brake release sample		0.4	once

The two air recordings are the author's own, each cut into a start, a loop and an ending. The compressor's stop is read from the game's `shared_sound.pak` (a 5.8 GB zip; only stored entries can be read); without it a made puff stands in. Every sound is brought to the same loudness, RMS 0.1, with peaks held to 0.6; the volume is `SoundVolume`/100 × 0.22, set by ear in the game at 50. They play through XAudio2 2.9, only with the game window in front and a truck driven.

The shapes follow the change (`SoundShapeAt`, `sound.h:471-499`), with p the progress:

$$\text{air out: gain} = e \cdot \exp(-1.036\ p),\ p = \text{clamp}((2 - m)/2, -0.5, 1) \tag{31}$$

$$\text{air in: gain} = e\ (1 - 0.3\ p),\ \text{pitch} = 1 + 0.08\ p \tag{32}$$

$$\text{compressor: gain} = e^{0.8},\ \text{pitch} = (0.5 + 0.5\ (1 - (1 - e)^2)) \cdot (1 - 0.07\ p) \tag{33}$$

e is the voice's envelope and *m* the pressure in modes (equation 4). Air out is 9 dB quieter at Low than at Normal, as in the recording, about 1 dB a second; air in rises a tenth in tone, as the weaker of two bursts does in the recording; the compressor's progress is the tank's level, (level − 0.3)/0.7.

The compressor is made from scratch at 44.1 kHz: 47 strokes a second, each a knock (rise 1.3 ms, fall 3 ms, timing jitter 1.2%, strength ±8%) through six ringing band-passes at 94, 193, 310, 640, 1250 and 2300 Hz, intake air as noise falling off through each stroke, and a hum at the 2nd and 7th harmonics. The puff is a second of shaped hiss: band-passed noise at 4500 and 1600 Hz, falling off with a 0.2 s time constant.

45. Limits and known issues

By design

- Made on the Steam build of 22 July 2026. The Epic build has not been tested; the Game Pass build is addressed by the 1.2.1 search but not confirmed. After a game update the mod may stand down until it is updated.
- Only the driven truck's wheels are changed; trailers keep their tires. Co-op has not been tested.
- Should the overlay not start (no swap chain found; the log says why) there is no panel or tab, the key cycles the modes and the pad is not filtered. Without a sound device the sounds are off.
- On the by-hand damage path the game's damage display follows at the next damage the game deals itself.
- The ground grip list only fills once the truck moves, so the balance waits for the first metres.
- The models are the mod's own reasoning: the scale, the rubber factors and the damage speeds have no measured source beyond what each section names.

Fixed after this manual's first version (1.3.0)

- **Cold weather flattened past what the game draws.** Below 15 °C Low's pressure falls under Low on the scale, and its offset is carried past 10 cm, up to 12 cm from about −18 °C down (default numbers). The room is shared out for the deepest mode's 10 cm, so before 1.3.0 equation (14) gave ×1.09 of the room at 0 °C and ×1.2 at −25 °C: on the test scout the tread sank into hard ground by about 6 and 13 mm, less the 0.005 *R* let through. Now no offset goes past the room.
- **A tire that comes soft went far too flat at Low.** Its contact sinks inside the cylinder, which FlattenLimit did not count; now the sink is measured and taken off the room (Tires that come soft).
- **The gear compensation fell short in the low gears.** Now each gear's top grows by roll exactly, equation (20).

Changed in 1.3.4

- Rolling drag (`RollingDrag`, the Realism section's "Rolling drag", on by default, untick it or set `RollingDrag=0` to turn it off): tires roll harder on soft ground and when aired down. Each wheel on the ground gets a rolling resistance by its ground and the pressure, through its own damping, as the game drags a flat tire. Mud counts as dirt for now. The weight is the truck's as the game's physics has it, its gravity included, without cargo or a trailer. Hard mode and the insanity mode turn it on. Hard mode's fuel factors are 1 now, as the engine burns more against the drag by itself.
- The insanity mode's fuel factors are 1, as rolling drag now does what they did.
- While rolling drag is on, every mode's fuel factor is 1: the drag makes a soft tire cost fuel by itself. The Fuel use row keeps its value and counts again with rolling drag off.

- The panel's pad and key controls wait while the game's function menu is up. The mod then hides no button from the menu, so the d-pad and A work it with the panel open.
- The last start's log is kept as `TirePressure.prev.log`.
- `TirePressure.log` names an invalid parameter in the C runtime and abort, which can end the game at once (0xC0000409). With Next Gen Gameplay in, its log names the game's and this one names the mod's own.
- The ASI loader starts on Linux (Proton and the Steam Deck) with the launch option `WINEDLLOVERRIDES="version=n,b" %command%`. Before, the mods never loaded there, and with that option the game would not start.
- With Spotter installed (its draw API, version 1 or later), the overlay draws through Spotter's frame hook instead of a hook of its own, where ReShade has not taken it as an add-on. `NextGen.ini` `[Overlay]` `Spotter=0` turns it off. While the NEXT GEN menu is open, the game gets no mouse or keys and Spotter's hotkeys rest. While Spotter's own menu or big map is open, the menu's key does nothing.
- Nothing about ReShade is shown on screen any more: the notice in the NEXT GEN menu and the line at the top for 15 seconds are gone. The log says which route the overlay takes. ReShade's `dxgi.dll` and `ReShade.ini` stay in the zip for RivaTuner users, whose game does not boot without it. The mods draw without it.
- The pressure mode is kept per owned truck, by the game's own id for it (read as Next Gen Gameplay reads it, so both mods name a truck alike). Two trucks of one kind each keep their own. `TirePressure.trucks.ini` holds `[kind:{id}]` with `Own=1` for one truck and `[kind]` for its kind. The mode in force is the truck's own, else the kind's, else the start mode. A truck not driven before starts in the highest mode on offer: Increased, or Normal with `Increased=0`. Under "Each truck keeps its own pressure" the tab has "This truck only". Ticked, a mode chosen is this truck's alone. Unticked, it goes to the kind's section and the truck's own is kept with `Own=0` until it is ticked again. When the game makes a truck anew under another id (a recovery, a garage), the new id takes the old one's mode over, as Next Gen Gameplay does, with one line in the log. A file from 1.3.1 to 1.3.3 reads as before: its trucks go by their kind until This truck only is ticked.

Changed in 1.3.3

- The step bindings are the plain d-pad left and right, `DPadLeft` and `DPadRight` by default (an ini's `LB+DPadLeft` becomes `DPadLeft`). They act only while the panel is open and no shoulder button is held. LB + d-pad is for gears and never the panel's, open or closed. While the panel is open the game sees none of the plain d-pad, so nothing shifts from inside it (two testers' reports, 27 October).

Changed in 1.3.2

- The overlay copies a swap chain's method table that another program has put at the start of its memory. Before, it drew nothing there (no panel and no NEXT GEN menu on two players' PCs). When a table still cannot be copied, `TirePressure.log` says in whose code the table and its Present lie, so the program can be named.
- LB + d-pad left or right works only while the panel is open. The two chords no longer open the panel a step lower or higher on their own, so a chord hit while driving changes nothing, and the game keeps those buttons while the panel is closed.
- A worn out or broken tire keeps a slight grip, `WornGrip` percent of its own on the ground, on asphalt and in mud, 10 by default, so the truck still moves; 0 gives the old none (the testers' ask). Hard mode and the insanity mode take 10 too. The tab has it as Worn tire grip.
- The pad bindings rest while the NEXT GEN menu is open, as the panel key does. A binding held as the menu opens or closes is no press when it closes.

Not changed, on purpose or not yet measured

- **Above Normal the Increased numbers count even when Increased is off.** In hot weather Normal slides toward them (equation 3), as over-inflated tires do; in the insanity mode the numbers used are its hidden Increased ones.

- **The pull of a flattened wheel** depends on how the game turns torque into push (see How a flattened tire rolls): worked out, not measured.
- The pressure step and the air tank count 0.05 s per pass, not the measured time; the stand-still test is one pass late; the two air sounds have no made stand-in if their recordings fail to load.

Where the README is looser than the code

- A change can take longer than the "up to 6 seconds" of the README: Low to Increased takes 8.7 s at the default.
- Hard mode's offsets are 5 and 2.5 cm only on tires with the room; equation (13) shrinks them on others, and the cold carries them further.
- The balance compares with the best vanilla tire of any kind, not of the tire's own kind.
- "Grip stops at 10" holds for the multipliers and the ground grip list; the asphalt and mud values written to a wheel are not capped.

46. The offline tests

`test.bat` builds and runs them without the game:

- The loader: the proxy is the one in use, forwarded calls return the system's answer, a test `.asi` is loaded; the same through a chain DLL ending in System32; and a module another loader already loaded is left alone.
- The ini: a missing ini is written with the defaults; every key reads back as written; help text and unknown keys survive; "nan" and "inf" give the defaults; old `Seconds=3` and `ConfirmSeconds=15` move to 6 and 5 once.
- Flattening: FlattenLimit within 2.5 mm of the hand-worked values for five radii; the room shared out as equation (13) says.
- Rolling: ×1 at stock, ×1.117 at the scout's Low, 95.1% of the speed kept; the gear numbers give each gear's top ×1.117 exactly.
- Cold: a 12 cm Low gets no more than the room, on the scout and on a 0.80 m tire with room to spare.
- A soft tire: a contact 3 cm inside the cylinder settles at 3 cm and comes off the room; a firm tire's takes nothing, and readings outside the window are not taken.
- The scale: each mode's pressure gives the mode's numbers; half way gives the mean; half a step past each end, then flat.
- The temperature: 15 °C changes nothing; Normal holds 0.827 at −20 °C and 1.098 at 35 °C; every effect goes the right way.
- The balance: f at strength 1, f^2 at 2, in between at 1.5; a vanilla tire stays at 1.
- Hard and insanity mode: each sets its numbers and options and leaves the key, volume and AmbientC; the insanity mode wins over hard; both turn rolling drag on, with fuel ×1.
- Rolling drag: at 10 m/s on paved at Normal the written word's torque within 1 % of Crr·N·r, with N by the Havok mass and not the files'; the same at three times the spin; under the floor the torque falls to nothing; ice none, sand at Low ×1.49, no flag dirt; a broken wheel left alone; airborne, off and another truck get the stock back; a write over 2 % and on the game's word; no climb over 10 lost records; none while the probe holds; the probe's stock back in the truck update; no hook, off.
- Shared state: the block is created, written and read back by name, every field at its offset.
- Sounds: WAV and ADPCM decoding, pak reading (zip and zip64), loop and ending cuts, made loops that join without a click and stay within level, the compressor starting on the right filling and stopping once, change times matching the passes.
- Game code, with a copy of the Steam image: every place found at its known address, and each of the three ways to the control global alone gives the same one.

47. Constants at a glance

Table 29. Every constant of the models

Constant	Value	Unit	Where it comes from
kModePressure	0.45, 0.65, 1, 1.25	share of Normal	Normal 35 psi; the rest the mod's choice
Factor ranges	grip, fuel, steering 0.1 to 10; offset 0 to 0.25 m		Expeditions' ranges for its keys
Scale overrun	half a step (t from −0.5 to 1.5)		the mod
Pass	50 ms sleep; 0.05 s step		"20 times a second"
Seconds	6 (hard 30, insanity 25)	s, Normal to Low	Expeditions takes 3; 6 since 1.1.0 for the sounds; hard: a truck's compressor
ConfirmSeconds	5	s	1.1.0
Stand still	0.3	m/s	"about 1 km/h"
Still detection	under 0.01 mm a pass for 4 passes		a paused game keeps its velocity
Rolling for fuel	on above 0.5, stays above 0.25	m/s	Expeditions applies its modifier above a speed
Air tank	use 0.17 a mode, start below 0.45, refill 0.04 a second	of a tank	not stated
Gas law	atmosphere 0.42 of Normal; reference 15 °C (288.15 K)		Normal 35 psi over 14.7
r_hard	−0.006 a degree below 15, +0.003 above, 0.7 to 1.08		"a fifth less at 20 below"
r_soft	−0.003 a degree below 15, at least 0.85		not stated
r_speed	+0.006 a degree below 15, 0.75 to 1.25		not stated
Temperature step	0.5	°C	no rewrite on drift
Damage speeds	Low 20 to 30, Reduced 35 to 45 (default)	km/h	Expeditions' off-road tires, lowered by driving on 2026-10-04
Damage	Low 4 to 8 a tick, Reduced 2 to 6	points	Expeditions' values doubled
Tick	Low 7, Reduced 10	s	not stated
Reset window	1	s	20 Hz polling against a check every frame
Wheel damage	about 50	points	the game
FlattenLimit	q = min(1.5h, 0.25R) + 0.02; pull 0.8 − 1.1 a/z; z = (min(0.2R, q) + 0.1) 0.8π/R; let through 0.005R		the game's wheel update and wheel shader
Cylinder floor	0.4	× R	"whatever the ini says"
Sink	blend 0.025 a pass (about 2 s); 5 mm steps, moved at 3/4 of a step; window 0.85 to 1.02 of the cylinder		the mod (1.3.0)
Gear numbers	a′ = roll·a + 2.5(roll − 1); high gear 2(roll − 1)		the game's 2a + 5, as the Gauges mod reads it
Havok half float	× 1.003906, upper 16 bits		the game's rounding
Stock acceptance	R 0.05 to 5 m, own offset ≤ 0.3R, asphalt (0, 10.5], mud [0, 10.5]		what a tire file can hold
Wheels, gears	at most 64 wheels, 32 gear values		not stated
Search	120 looks, 500 ms apart; 10 equal looks for a partial result		Steam unpacks its code at start
Game thread	update seen within 500 ms; job dropped after 1 s		not stated

Constant	Value	Unit	Where it comes from
Pad rescan	2	s	not stated
Shared block	64 bytes; stale after 2 s; looked for every 5 s		the mod
Sound level	RMS 0.1, peaks 0.6, volume × 0.22		set by ear at 50
Envelope	16 of 441 vanilla tires (94 grip sets)		initial.pak, tools/vanilla_tires.ps1

48. Licence and credits

GNU General Public License v3.0 (GPL-3.0-only). Source: github.com/andizla/snowrunner-tire-inflation-system.

The tire inflation system, its panel and its numbers are from Expeditions: A MudRunner Game by Saber Interactive. The panel and the menu are drawn with Dear ImGui by Omar Cornut, and their text is baked with stb_truetype by Sean Barrett; ReShade by Patrick Mours (crosire), which the release brings along without its start banner (BSD 3-Clause licence), hands the mod the game's swap chains through its add-on API. The asphalt floor follows an idea by Naybour. The insanity mode is built from the settings of the player Globalhacker. SnowRunner and Expeditions are games by Saber Interactive; this project is not affiliated with Saber Interactive or Focus Entertainment and contains no game files.

A big thank you to the testers who gave their time to the early builds: Milos_Mk6, Globalhacker and Zenosxy. Your runs, logs, ideas and patience shaped this release, and it would not be what it is without you.